Agentic AI:
Essential Concepts for Builders

From LLMs to AI agents

Agentic AI
Module 1

Today’s activities

- Fundamental concepts of Agentic AI
- Agent implementation suitability
- Responsible AI for agentic systems
- Current agentic landscape
- Agentic design thinking

How to use this slide deck depending on your persona

This full slide deck can be split according to the following two personas:

 code-based faculty

 code-free faculty

- At the bottom left of each slide you can find the icons that apply for the respective personas
- You can use them as a guide to select the proper slides according to your needs
- The slides pointing to hands-on labs also have the same icons to guide you to select the ones that best fit to your faculty's persona

Possible combinations

- Slides for code-free personas: 
- Slides for code-based personas: 
- Slide for all personas: 

Code-free personas can skip the code-based slides without compromising the full understanding of the concepts covered

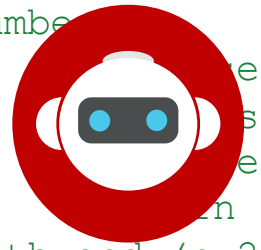
Agentic AI Fundamentals

From LLMs to AI agents

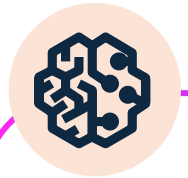
Key questions

- Can an LLM do **more** than producing text?
- How do you use an LLM for **real time information** and **database queries**?
- How can you **combine** an LLM with other systems and enhance the model's capabilities?

```
# Function to calculate the nth
Fibonacci number recursively
def fibonacci(n):
    """
    Calculates the nth Fibonacci
numbe                    es:
                         s 0 or 1, return n.
                         e case:
                         n the sum of the (n-
1)th and (n-2)th Fibonacci
numbers.
    """
    if n <= 1:
        return n
    else:
        return(fibonacci(n-1) +
fibonacci(n-2))
```



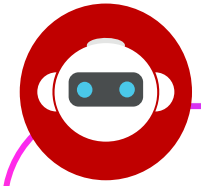
LLMs vs. AI agents



LLMs

Very large deep learning models that are pre-trained on vast amounts of data

May use **retrieval augmented generation (RAG)**, access tools or APIs



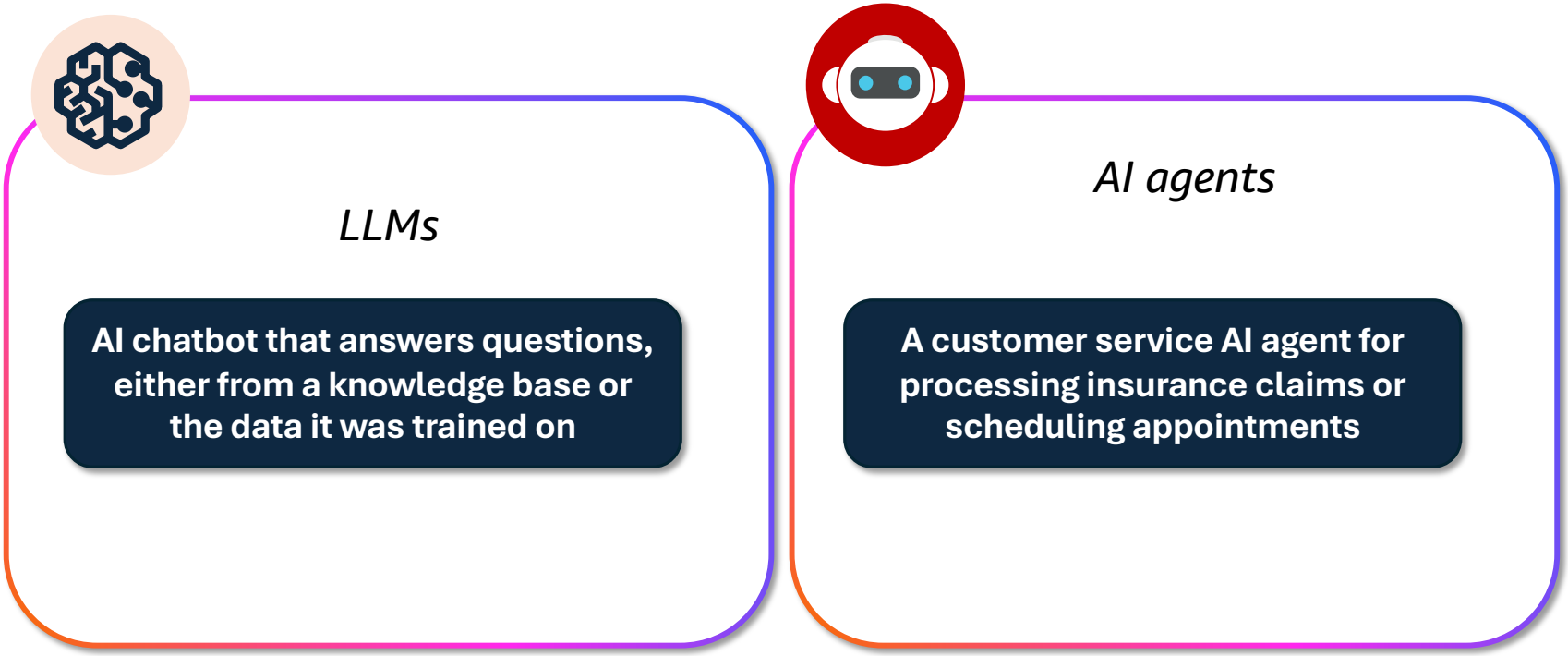
AI agents

Can **interact with an environment**, collect data, and use the data to perform self-determined tasks to meet goals

Humans **set goals**, but an AI agent independently chooses the best actions to achieve those goals

Access **tools** to execute **workflows**

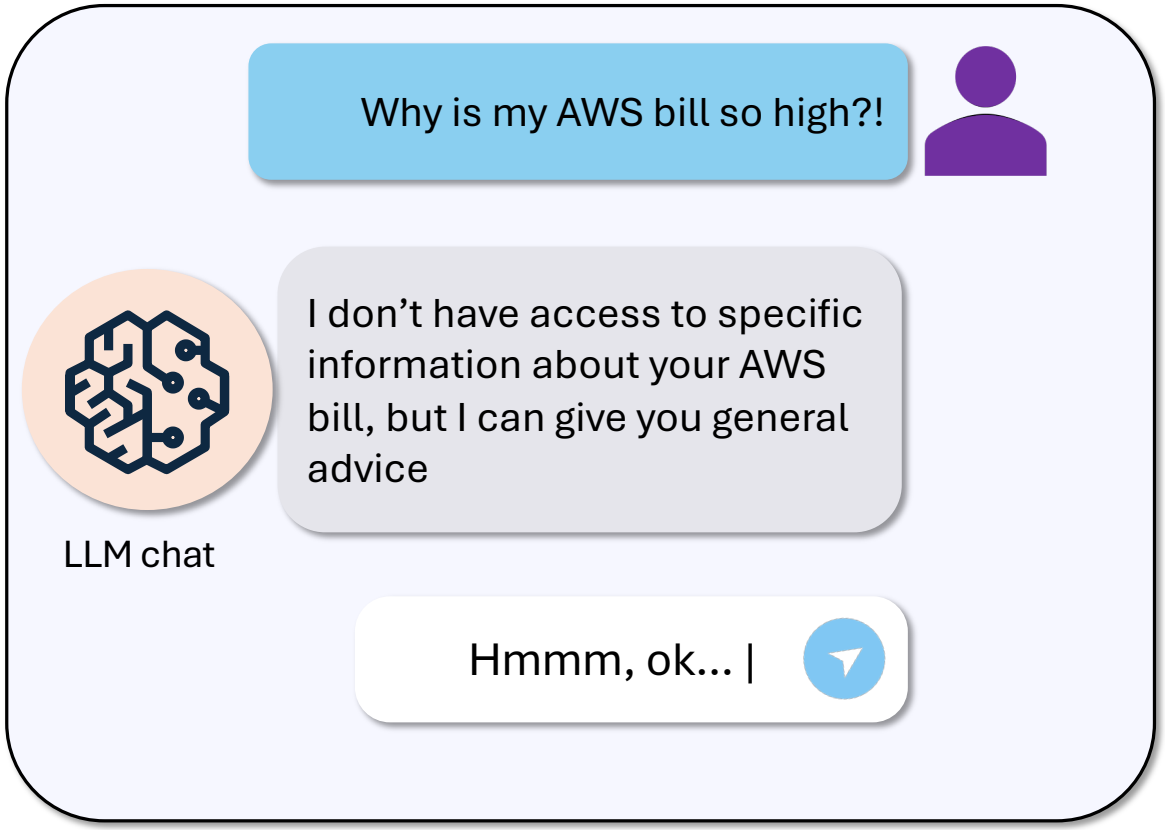
LLMs vs. AI agents examples



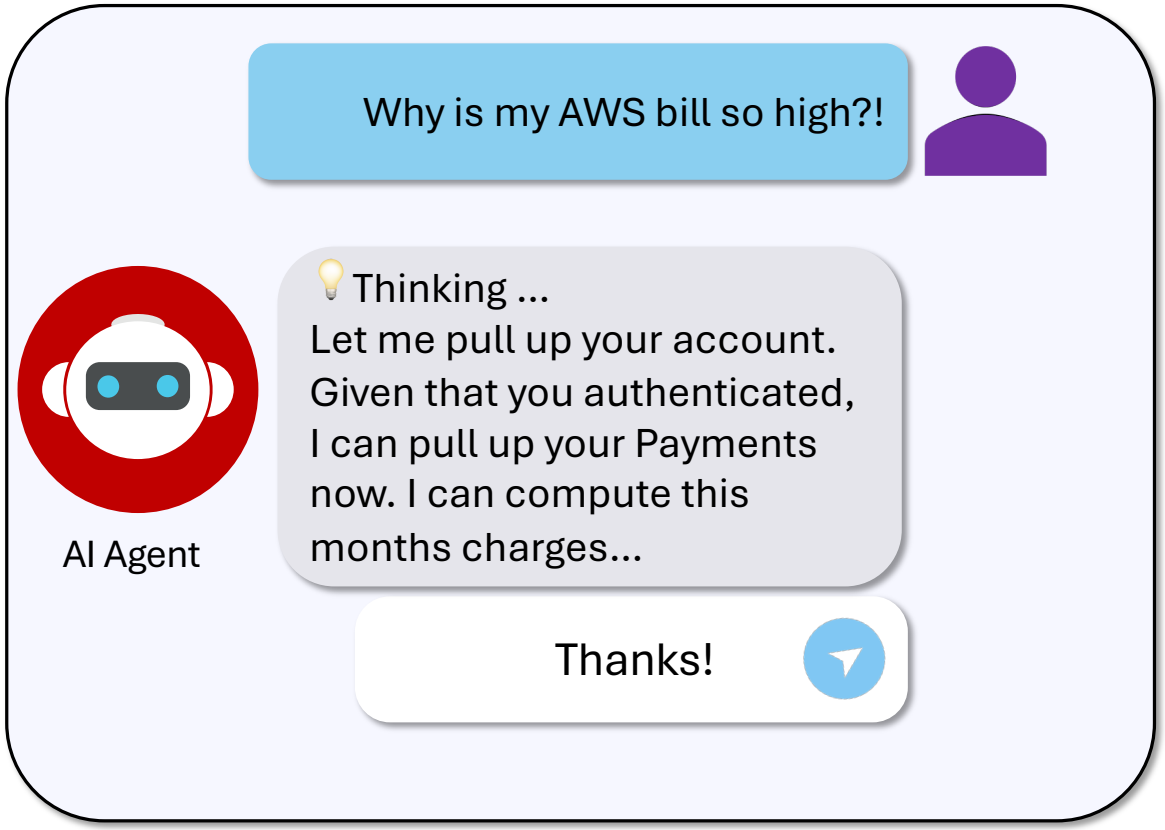
Agentic AI Fundamentals

What are AI Agents?

Let’s begin with an example



AI agent response example



Classic AI agent definition

Agent

noun

“An intelligent agent is an agent that acts appropriately for its circumstances and its goals, is flexible to changing environments and goals, learns from experience, and makes appropriate choices given its perceptual and computational limitations.”

Russel & Norvig (1995) - Artificial Intelligence: A Modern Approach, 4th ed., p. 34.

One Current AI Agent definition

AI • agent

noun

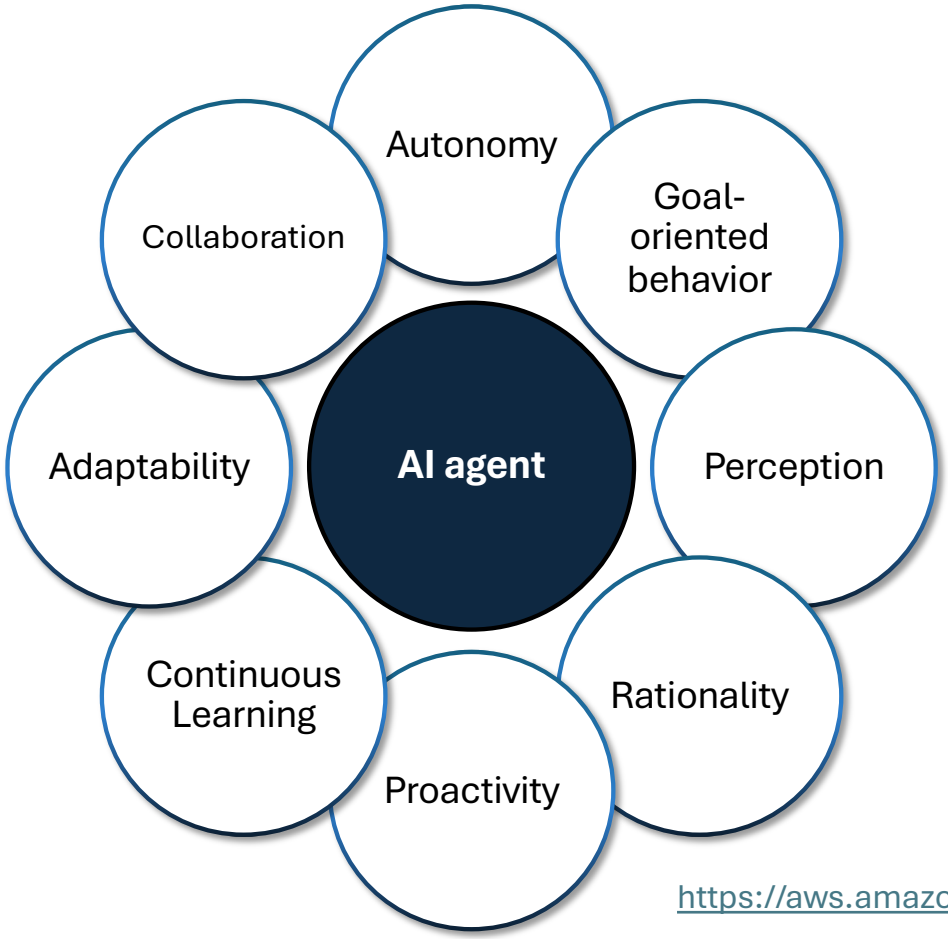
“An artificial intelligence (AI) agent is a software program that can **interact with its environment**, **collect data**, and use that data to perform **self-directed tasks** that meet **predetermined goals**. Humans set goals, but an AI agent independently chooses the best actions it needs to perform to achieve those goals”

https://aws.amazon.com/what-is/ai-agents/

AI agents are...

- Advanced AI systems that **automatically figure out how to best solve tasks**
- Powered by **LLMs** as their reasoning engines
- Capable of using **tools** to execute **workflows**
- Capable of following **guidelines/instructions** that define overall agent behavior

What are the key principles that define AI agents?



https://aws.amazon.com/what-is/ai-agents/

Agentic tasks

Knowledge-intensive tasks

Simple LLM prompting suffers from fact hallucination.

AI Agents may mitigate this risk by identifying the data sources, writing queries and retrieving relevant data

Decision-making tasks

LLM planning capabilities are enhanced by **custom tools**.

Selecting **appropriate tools** to **execute tasks**

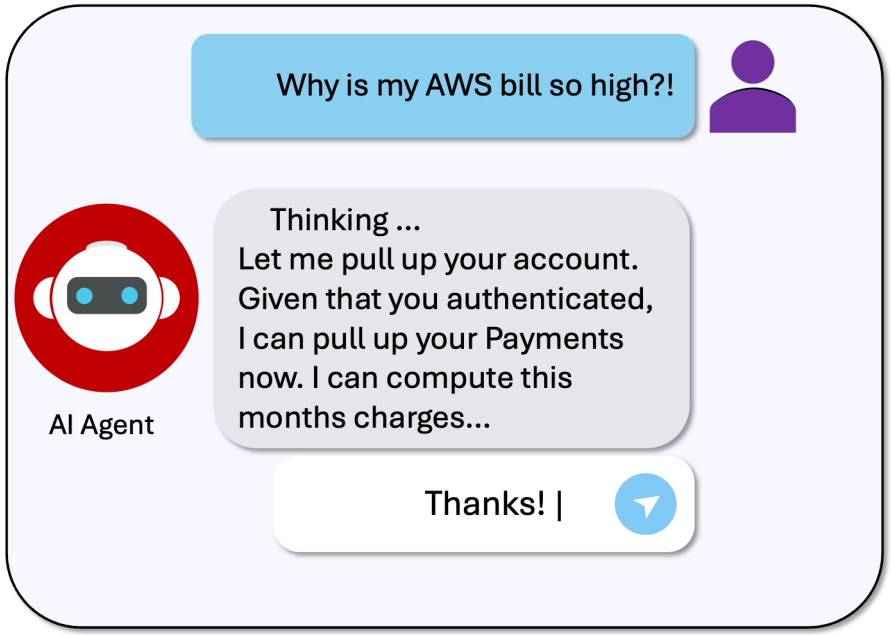
However, it's still far from the performance of expert humans.

What can AI agents do?

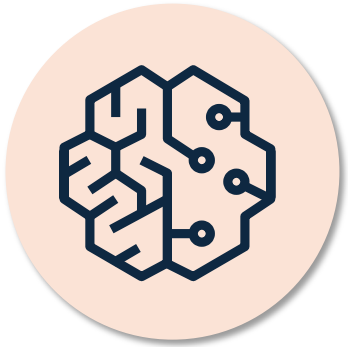
- **Understand** the request in natural language text using large language models.
- **Generate** a plan using Chain-of-Thought and ReAct (Reasoning + Acting) prompting.
- **Identify** required resources like APIs, data sources, and tools.
- **Execute** the plan using API invocation/Knowledge Bases.
- **Overcome** obstacles by retrying the process.

How do the steps apply to our example?

- **Understand**
- **Generate**
- **Identify**
- **Execute**
- **Overcome**



Let’s ask a chatbot about our costs



Why is our team's AWS bill so high this month?

MY ASSISTANT

Hold on, let me come up with a plan

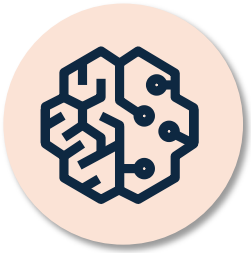
Let me check your internal data for any cost-related information while also loading AWS knowledge resources to help diagnose what's driving up your bill.

Searching your documents

Searching *"AWS bill high cost this month team spending"*

LLMs can *tell* you a lot...

Sure! I can give you some general suggestions on that topic!



Why is our team's AWS bill so high this month?

MY ASSISTANT

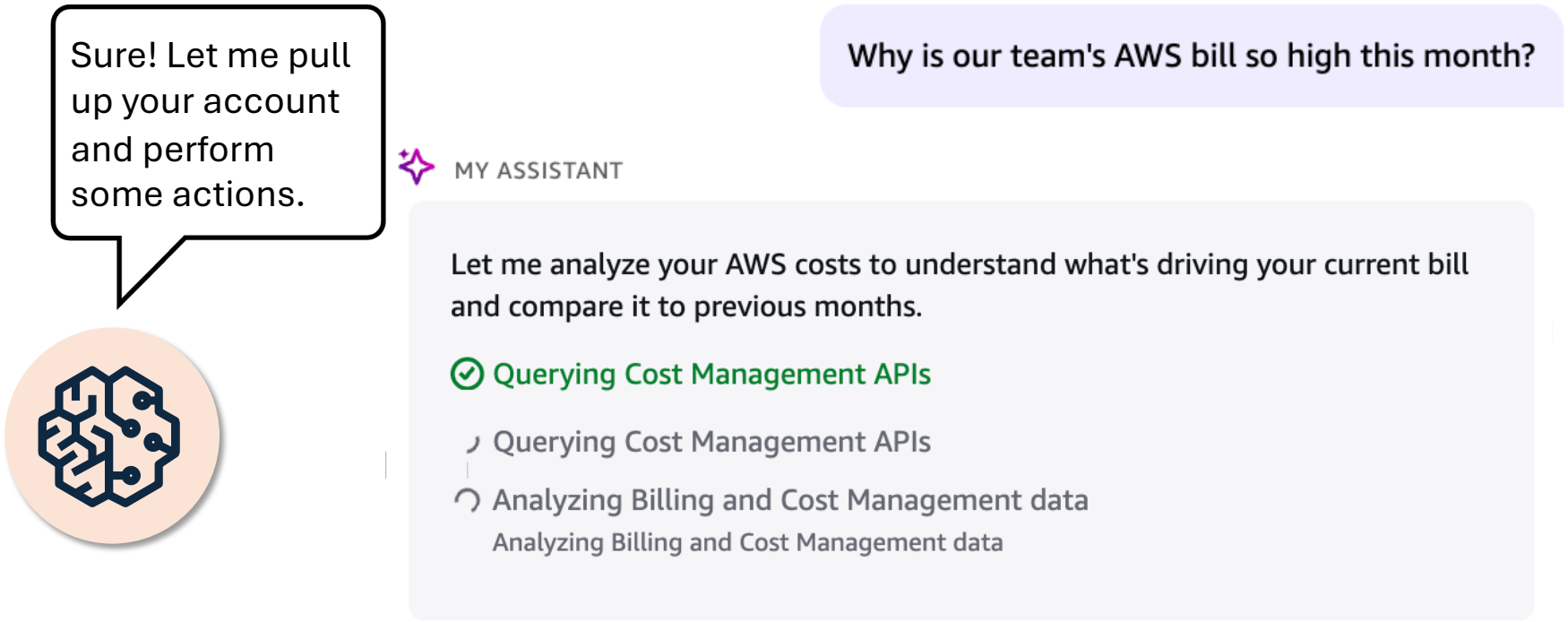
Response events

Based on your internal documentation, there are a few common reasons why an AWS bill might spike : 1 2

Common causes of high spend:

- Keeping resources running idle when not needed
- Reproducing issues on oversized resources (e.g., using a D2.8xlarge when a T3.nano would suffice)
- Launching too many instances when fewer would reproduce the issue
- Long-running resource usage without termination
- Using resources for unapproved personal or side projects

...But AI agents can *do* something about it.

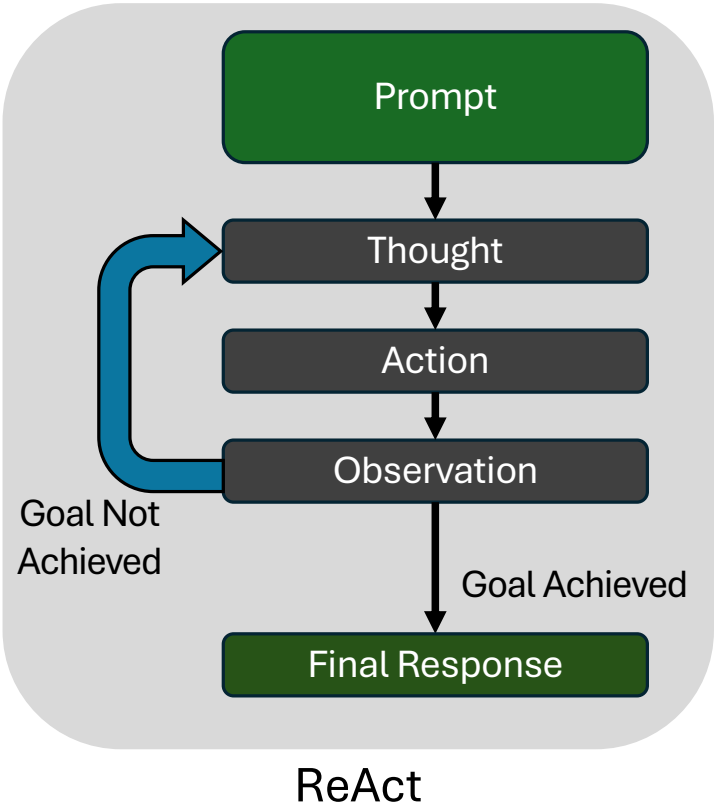


Agentic AI Fundamentals

AI agents with tools walkthrough

ReAct

- Interleave reasoning traces and task-specific actions
- Core technique used by most agentic frameworks
  - Interact with external tools to retrieve information or take actions
- Allows an LLM access to the external environment
  - Typically implemented as a cyclic process
  - Think one step at a time
  - Dynamically create and adjust plans

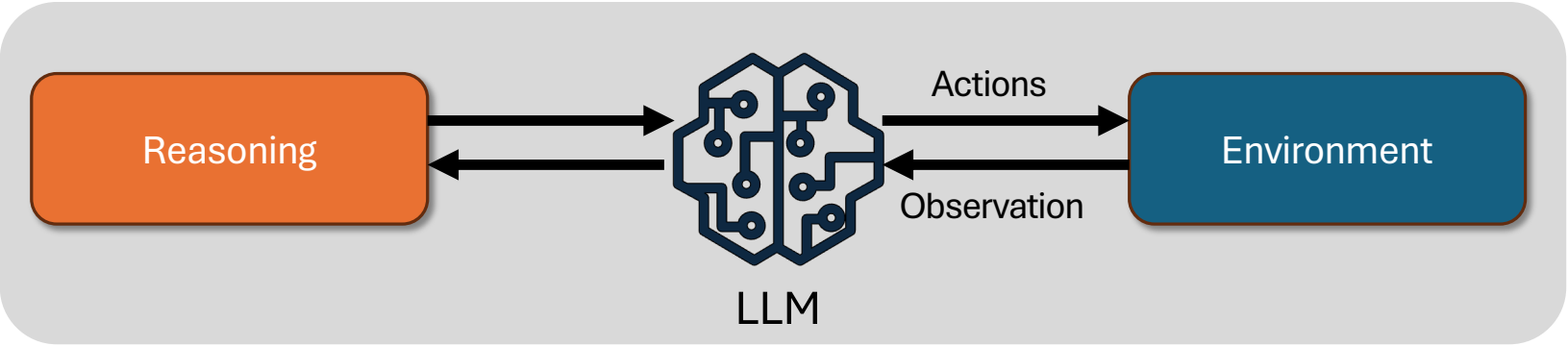


ReAct

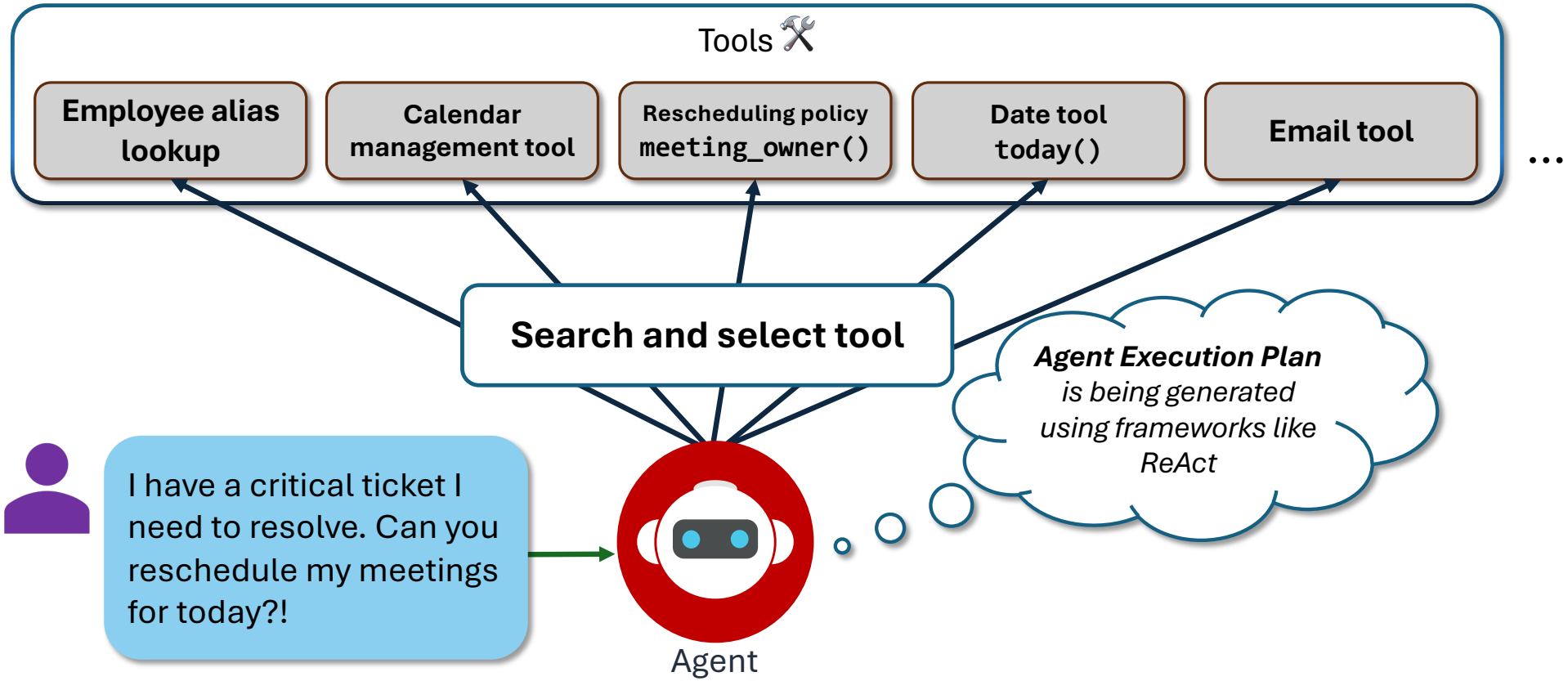
Source: Yao et al. 2023

ReAct: Reason + Act

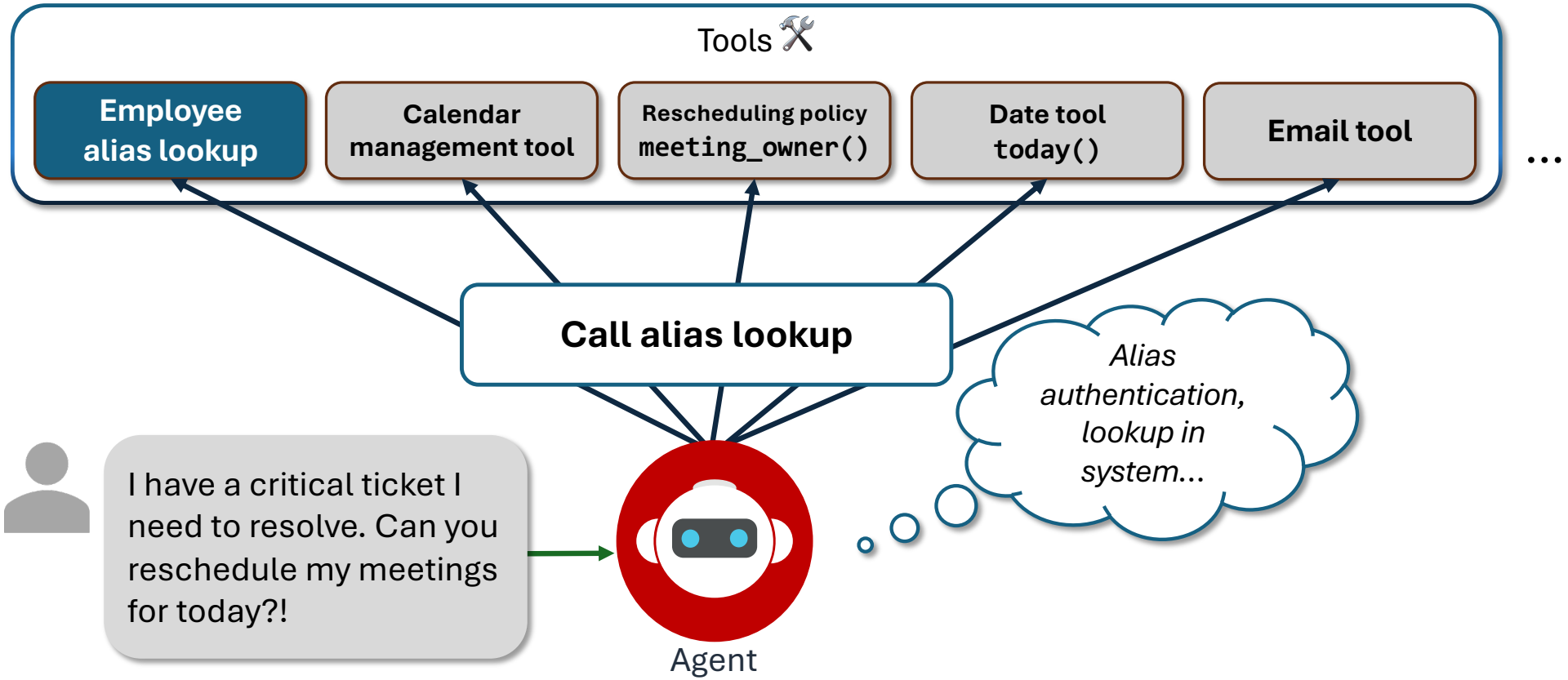
- LLMs are capable of reasoning and generating action plans.
- Combine both!
  - Reasoning: Create, track, and update action plans. Handle errors.
  - Acting: Interface with functions, tools, knowledge bases, or environments



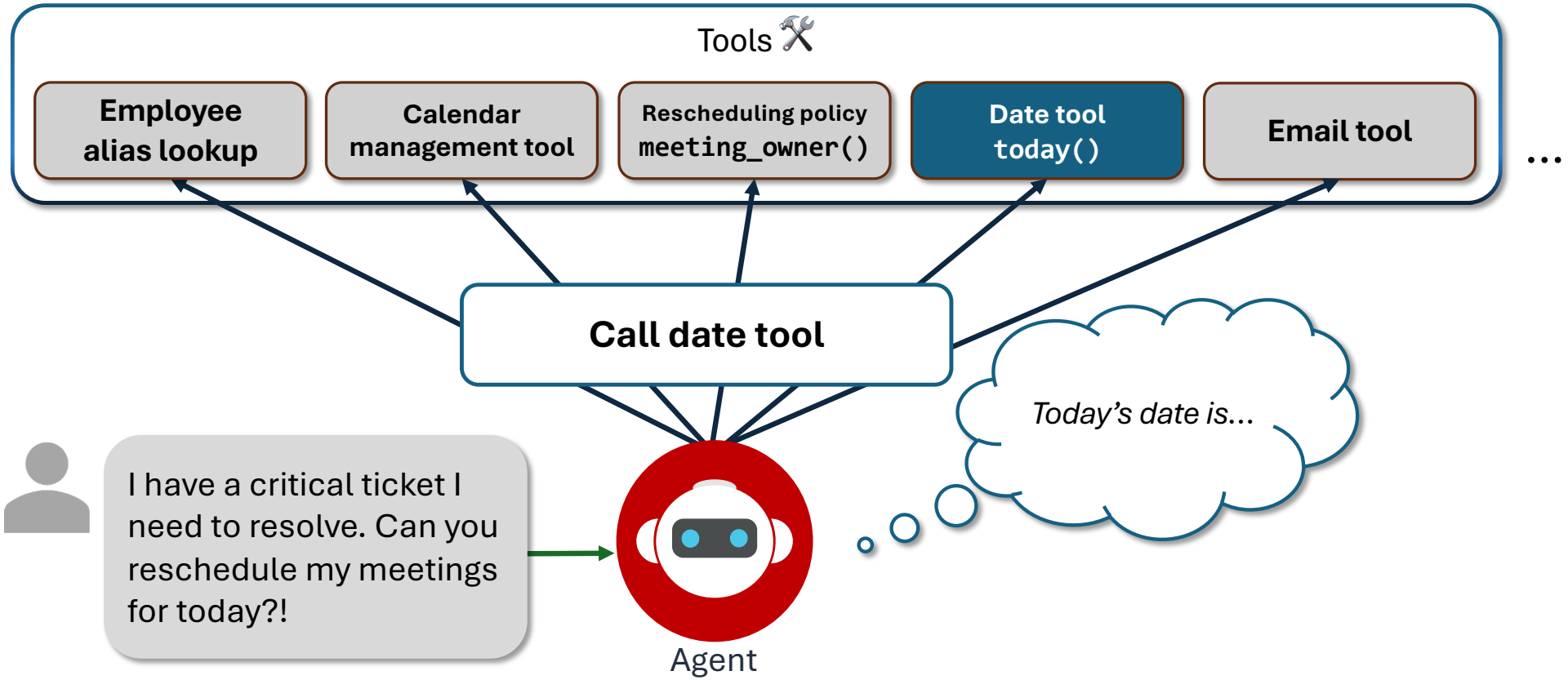
AI agents with tools – identify tool(s)



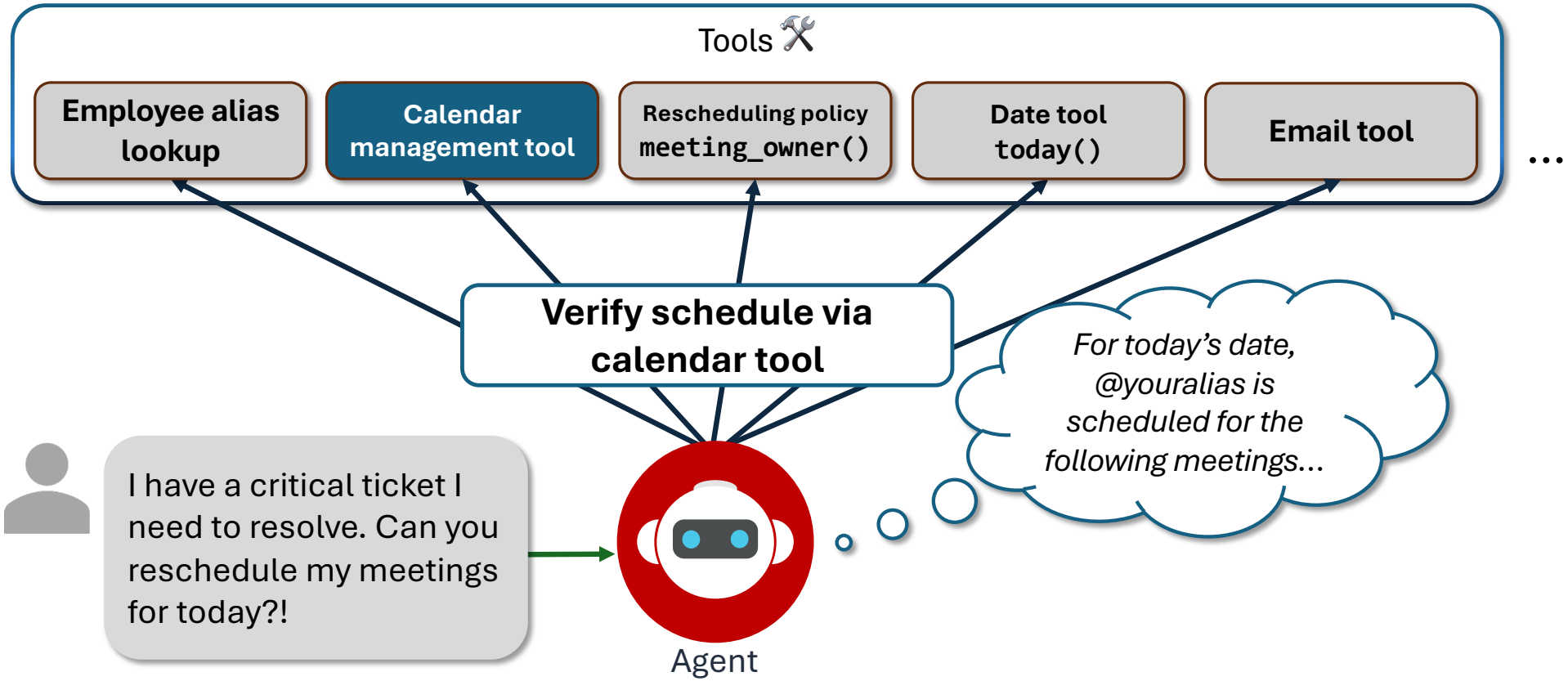
AI agents with tools – call alias lookup



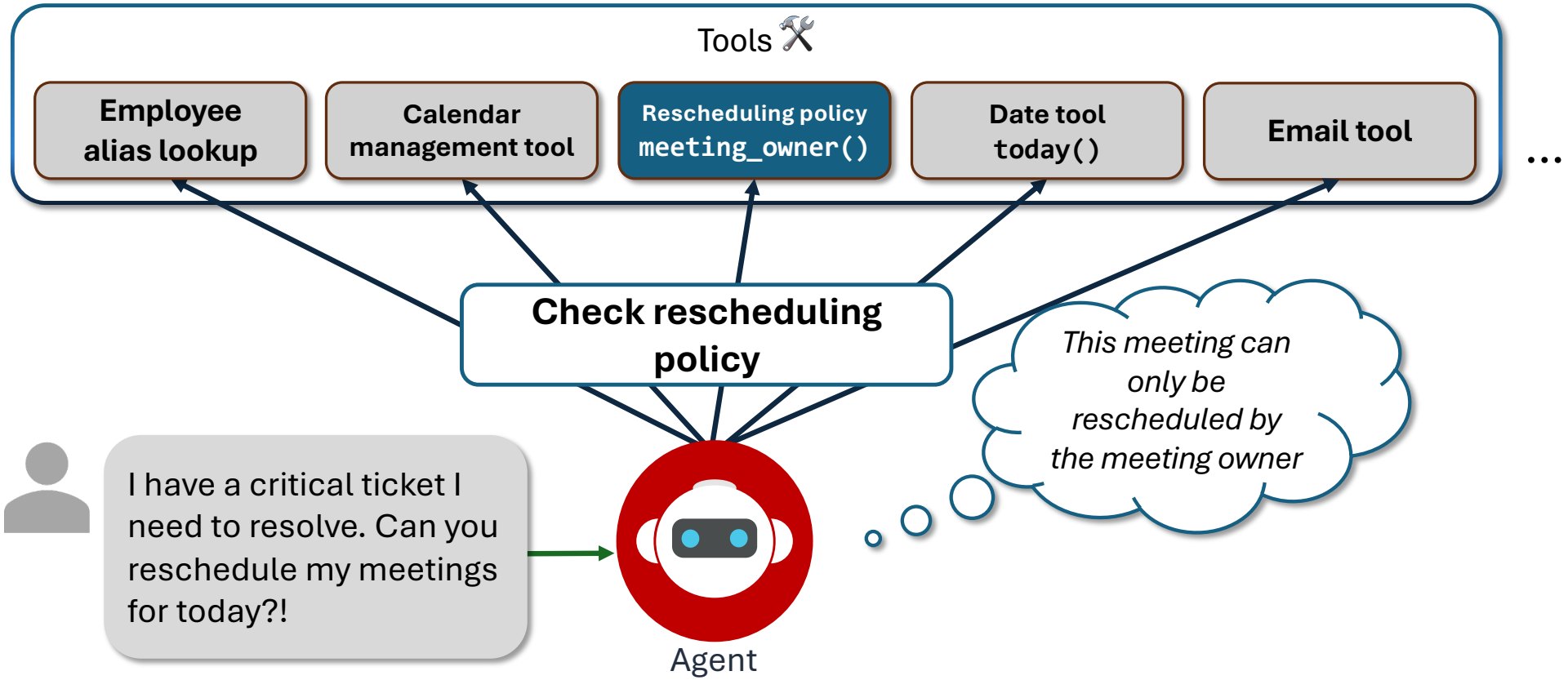
AI agents with tools – call date tool



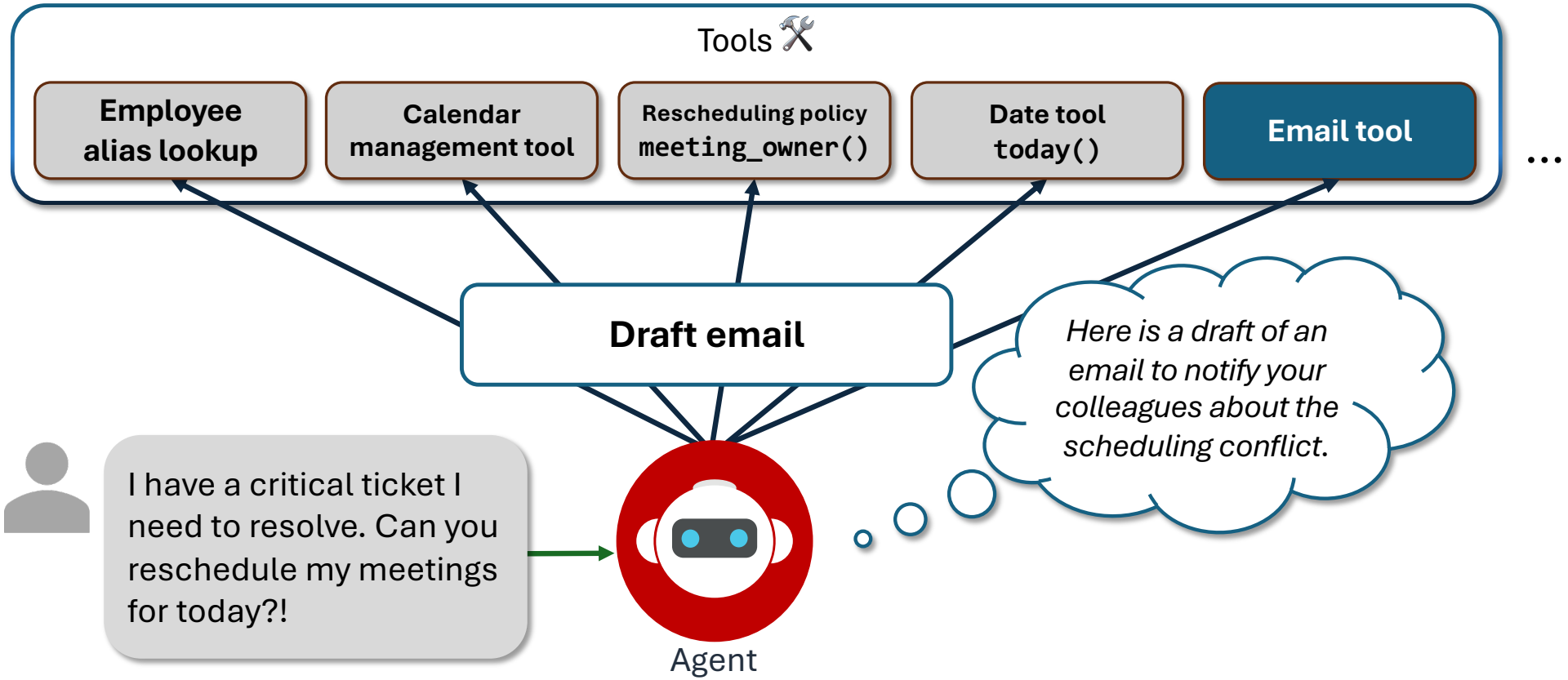
AI agents with tools – verify today's schedule



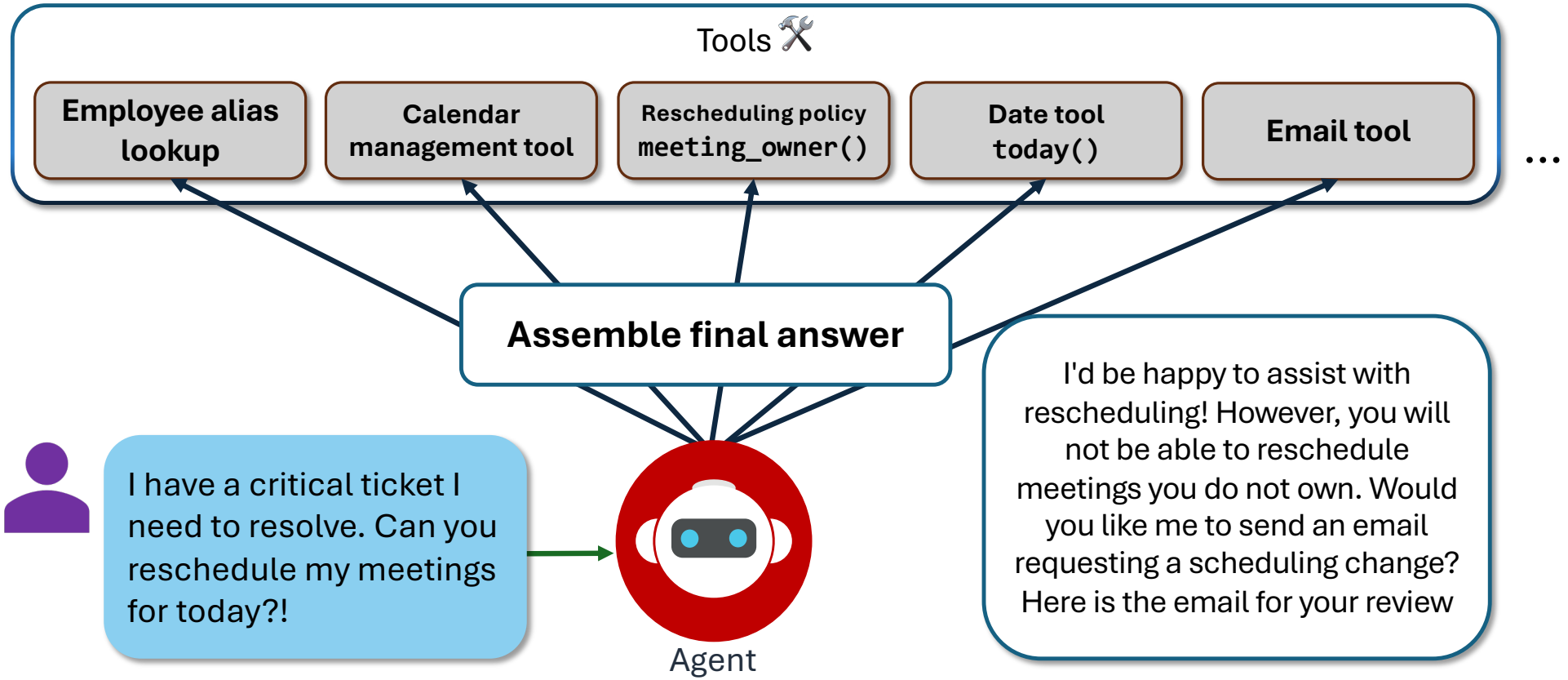
AI agents with tools – check rescheduling policy



AI agents with tools – use email tool



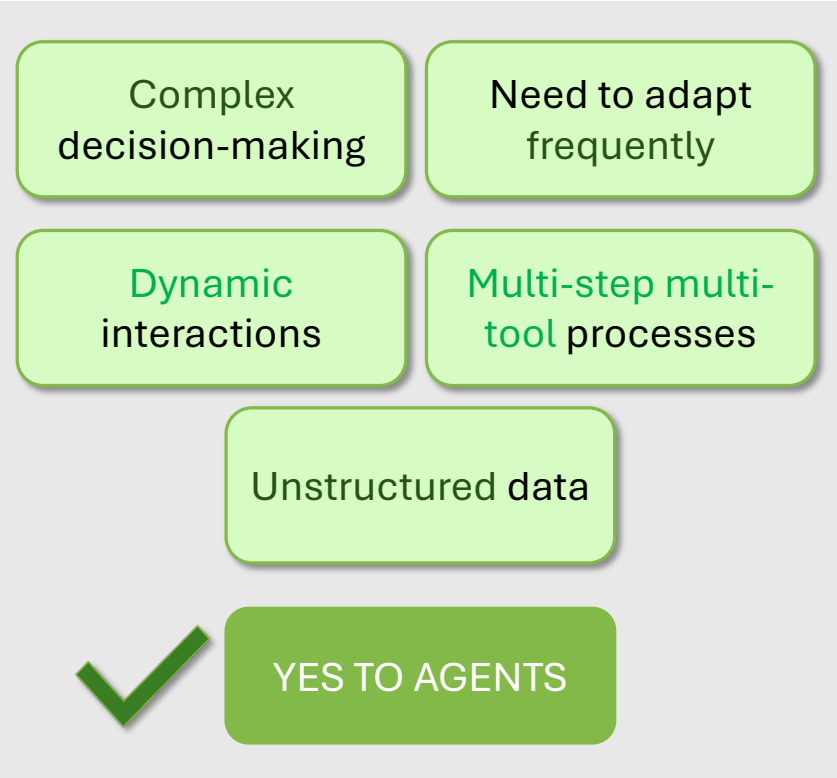
AI agents with tools – aggregate tool results

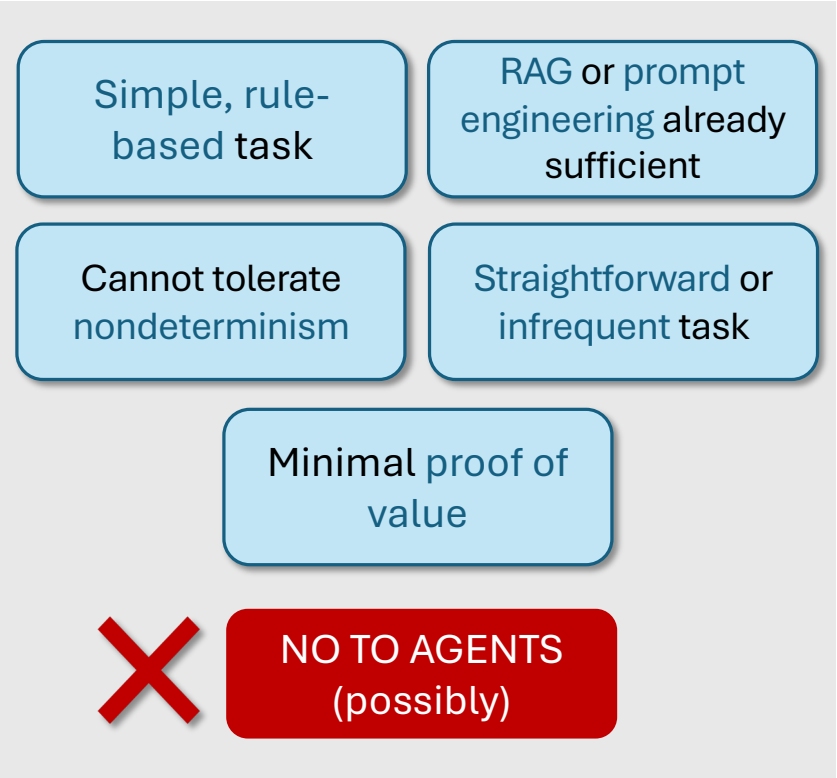


Should I build a solution using agents or not?

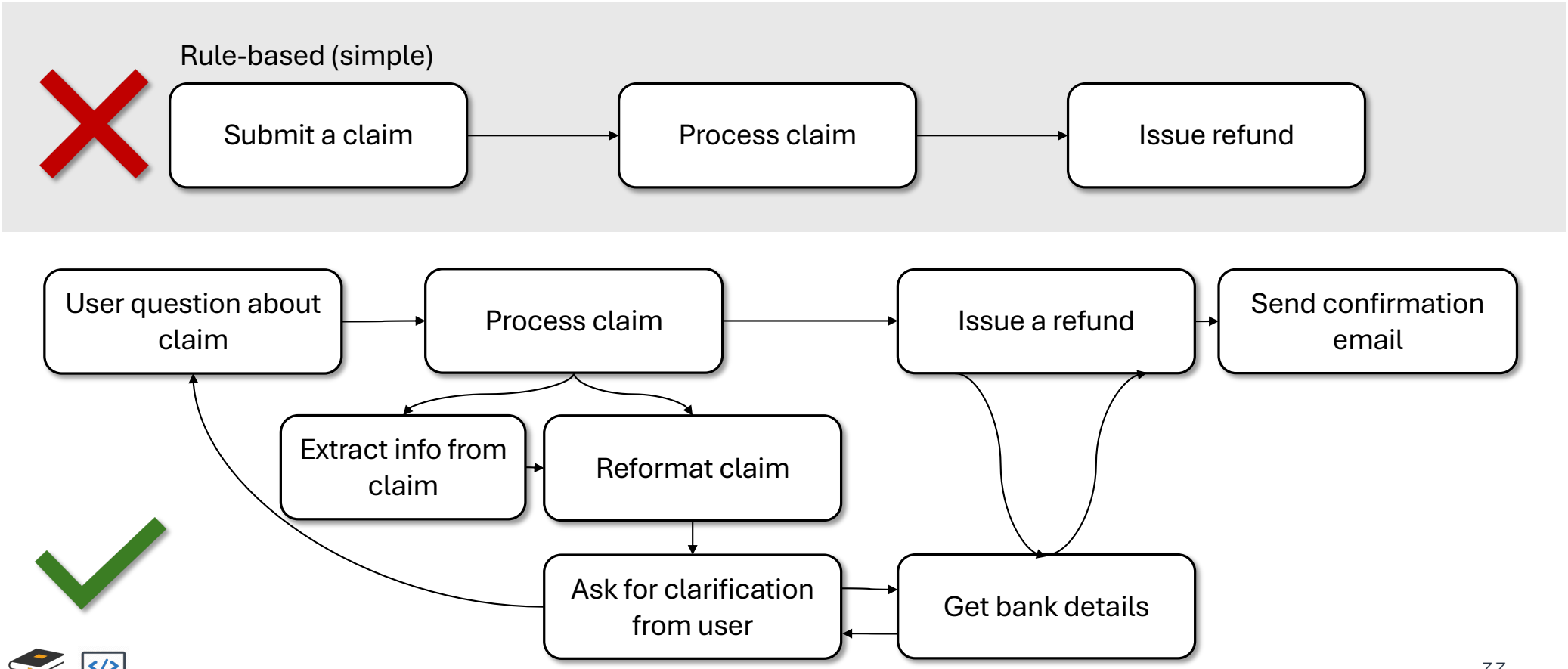
What are AI agents good at?

To agent or not to agent, that is the question

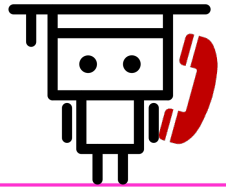




Decision-making beyond rule-based approach



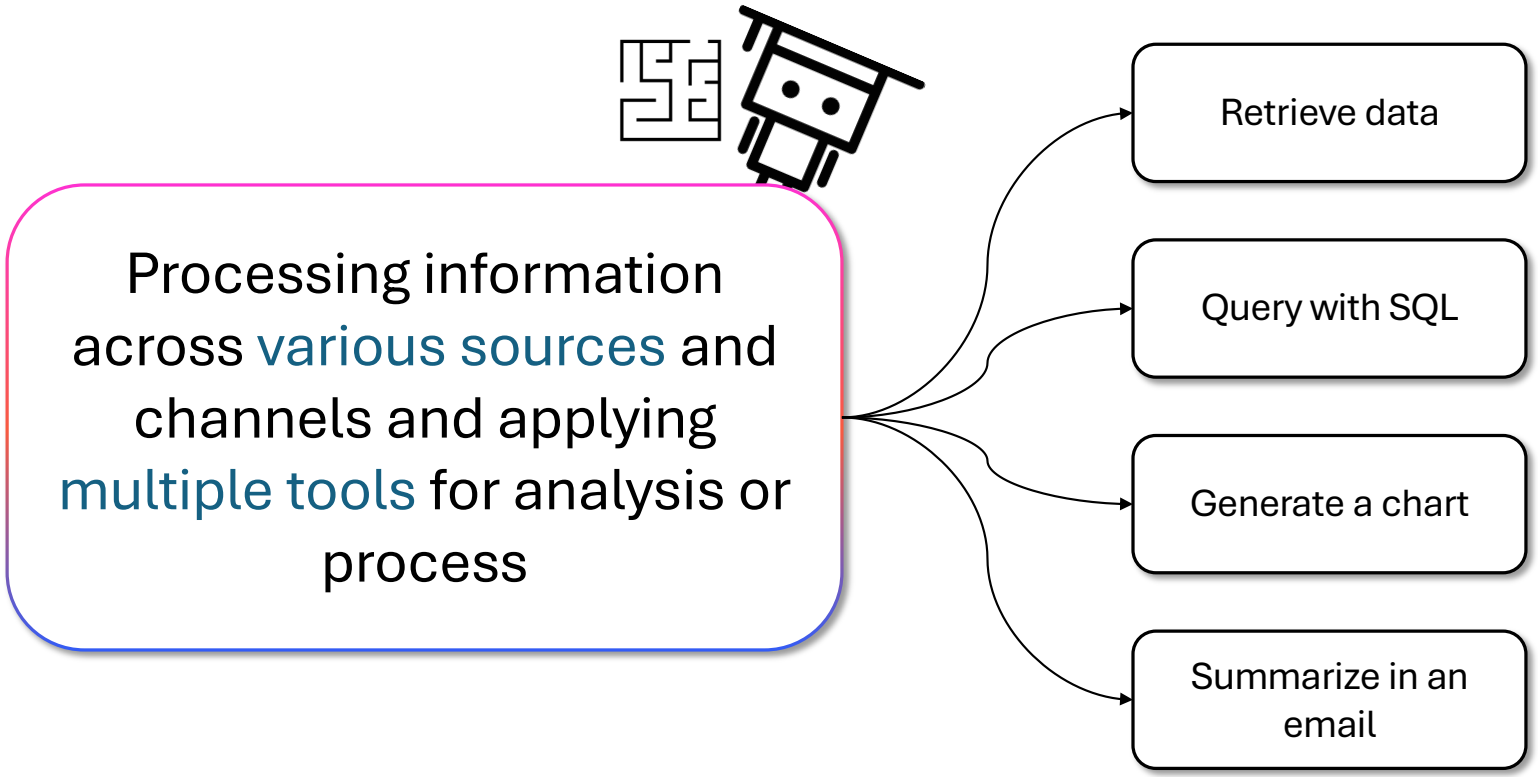
Dynamic customer interactions



Personalized customer service that needs to understand context and handle multiple topics or actions in a single conversation

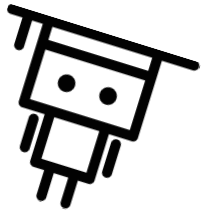
- Return my order
- General FAQ
- Change my profile information
- See my transaction history

Complex multi-step multi-tool processes



Natural language or unstructured inputs

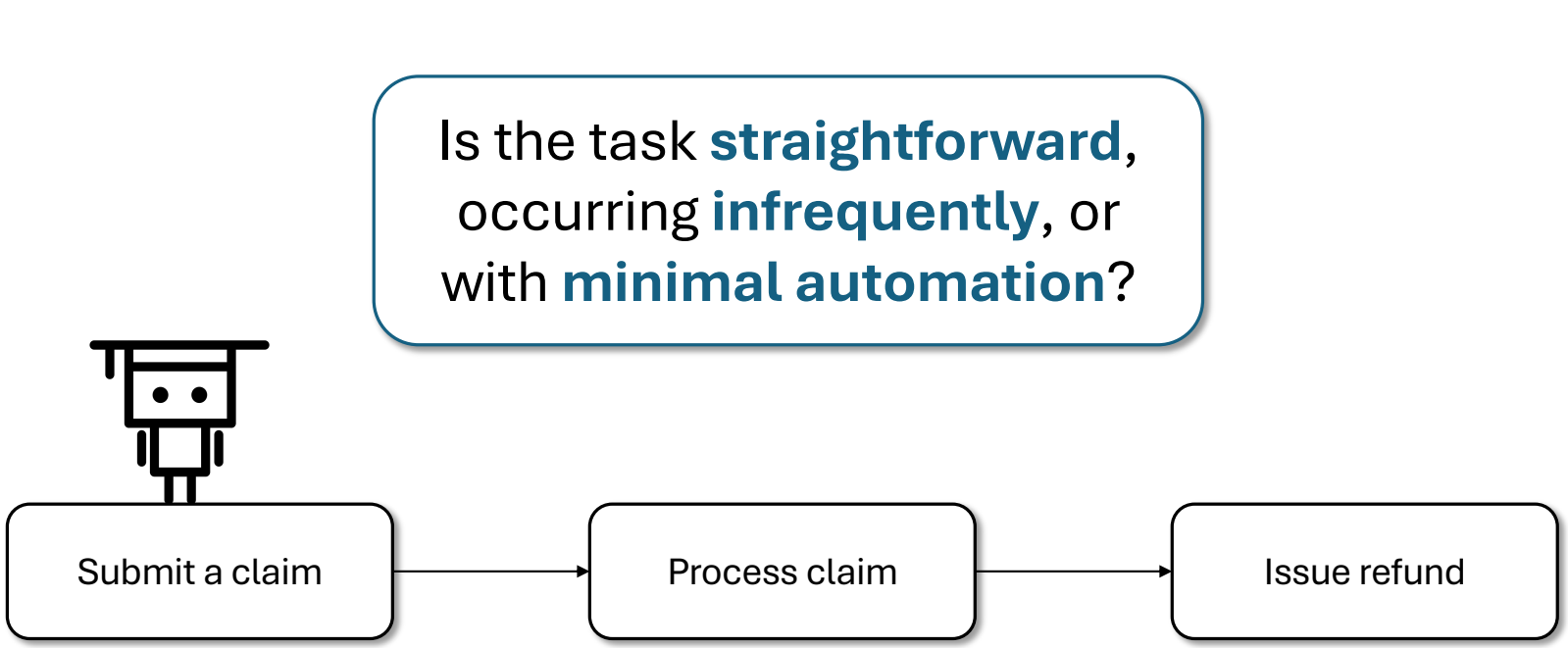
Handling and extracting insights from unstructured data (text, media, images, audio, etc.)



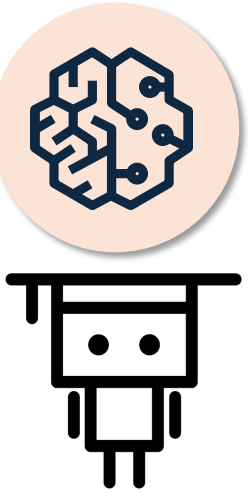
Should I build a solution using agents or not?

When are AI agents NOT the best solution?

Simple, rule-based tasks



RAG or prompt engineering is sufficient



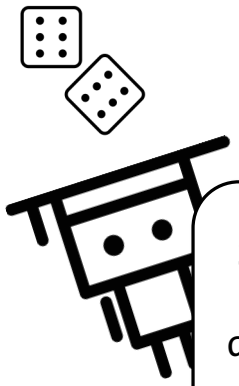
Can you solve your problem with RAG, few-shot prompting, chain-of-thought, or other prompt engineering techniques?

Basic RAG works well

Solvable with few-shot prompting

Chain of thought (C-o-T) is enough

Downstream tasks cannot tolerate nondeterministic outputs

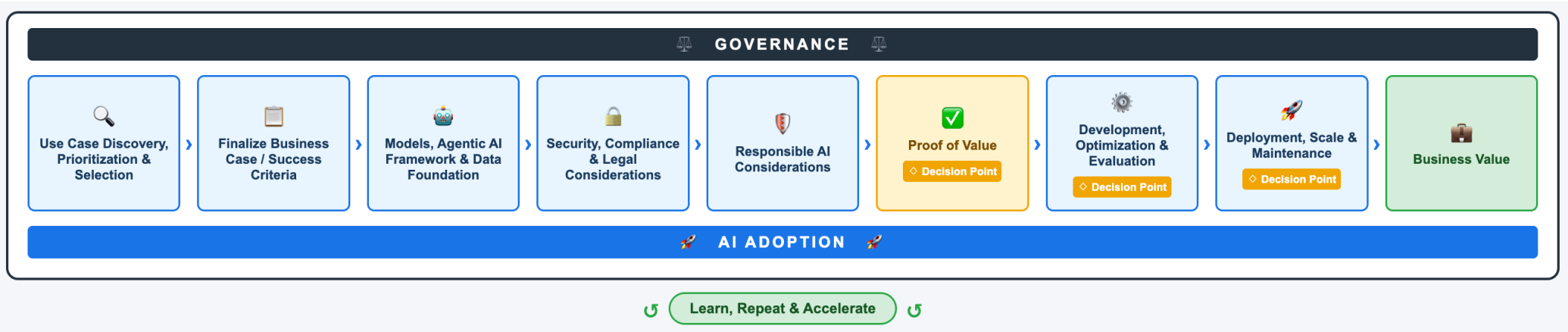


Can the downstream task tolerate nondeterminism? Is a high level of reproducibility and consistency needed?

```
SELECT first_name, last_name
FROM employees WHERE
department = 'Sales' ORDER BY
first_name, last_name;
```

```
SELECT first_name, last_name
FROM employees WHERE
department = 'Sales' ORDER BY
last_name, first_name;
```

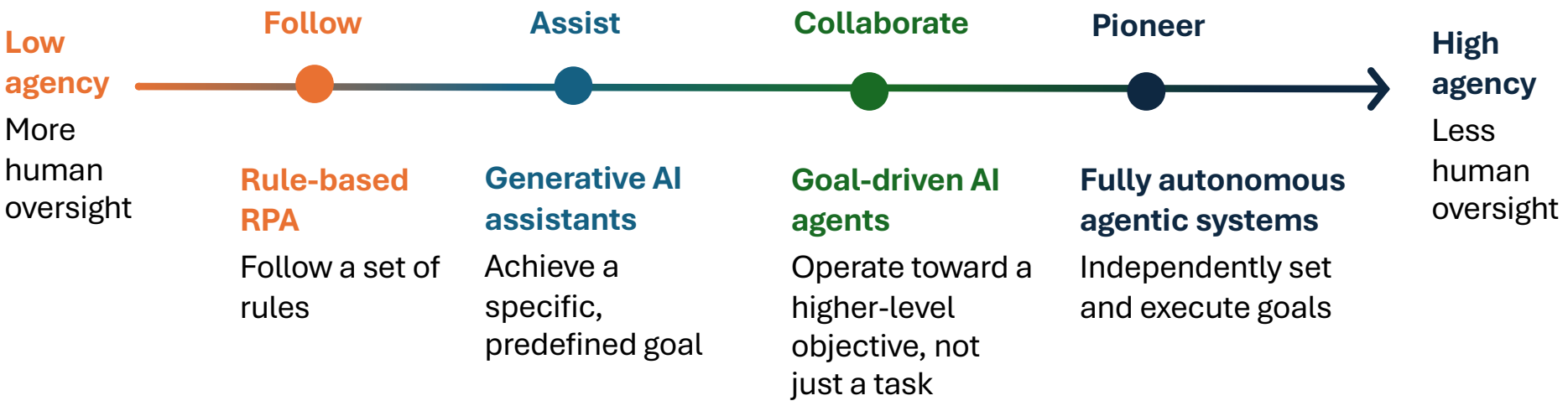
Agentic AI lifecycle



Responsible AI and Agentic Evaluation

Responsible agentic use

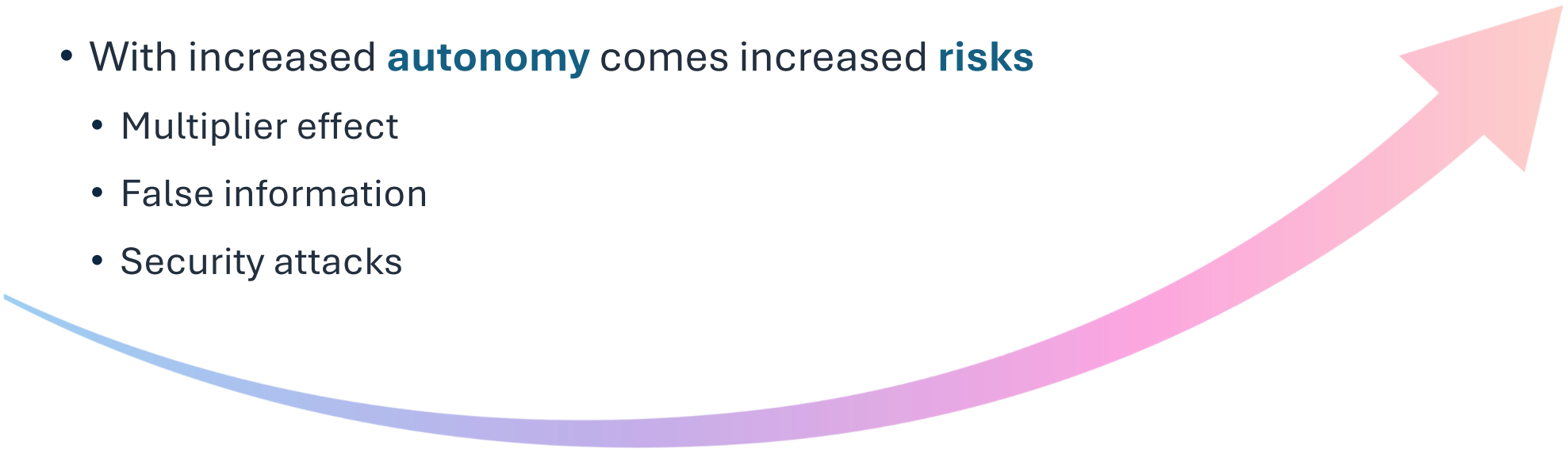
A spectrum of autonomy



Risks specific to agentic AI

- Agentic AI may pose unique risks with the ability to call tools, take actions, and execute code

- With increased **autonomy** comes increased **risks**
  - Multiplier effect
  - False information
  - Security attacks



Risk assessment guide

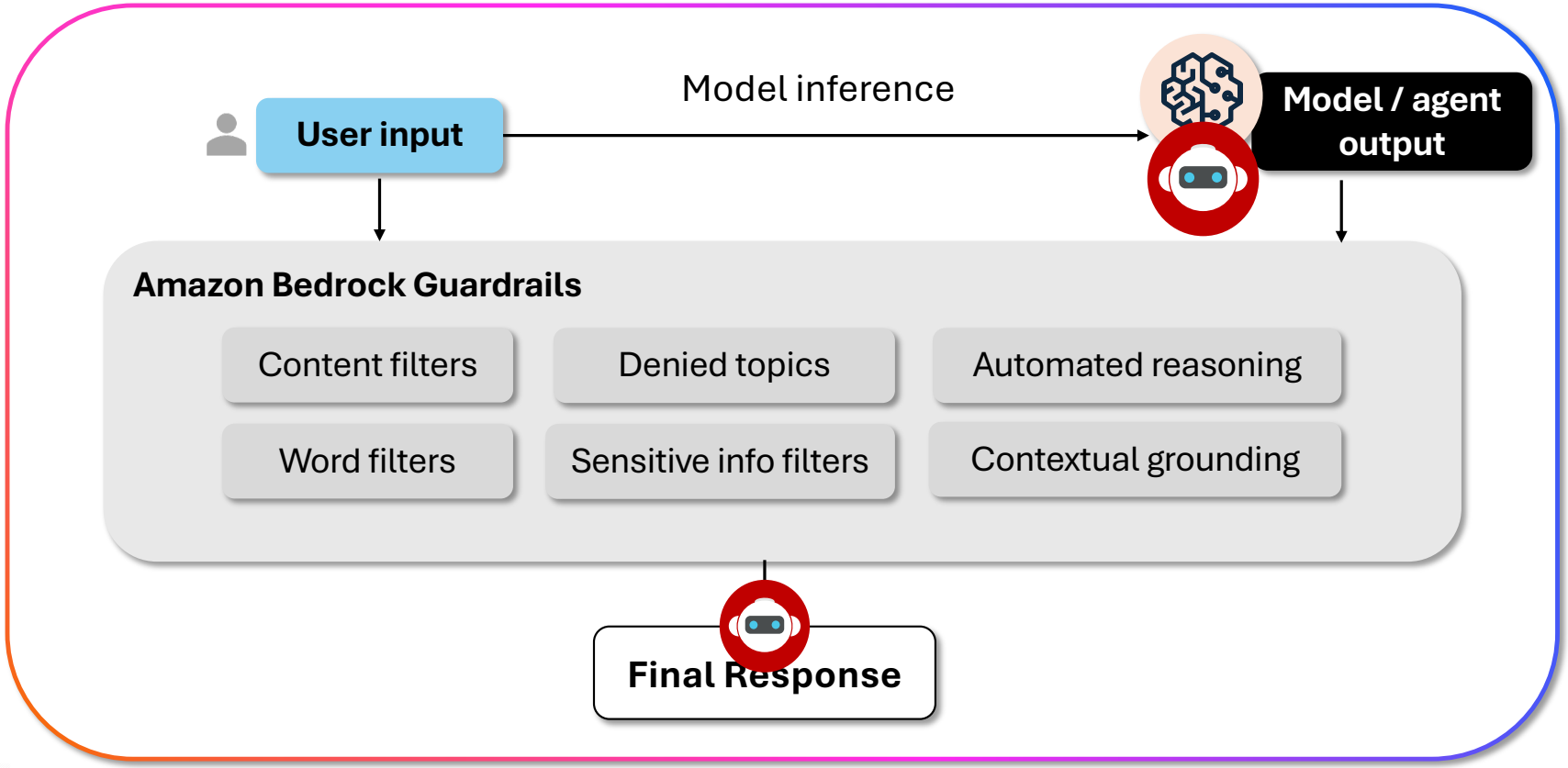
- **Risk** is a combination of **Likelihood x Severity** of possible harmful event

Severity \ Likelihood	1: Rare	2: Unlikely	3: Possible	4: Likely	5: Almost Certain
5: Extreme	Low	Medium	High	Critical	Critical
4: Major	Very Low	Low	Medium	High	Critical
3: Moderate	Very Low	Low	Medium	Medium	High
2: Low	Very Low	Very Low	Low	Low	Medium
1: Very low	Very Low	Very Low	Very Low	Very Low	Low

Responsible agent AI use

- Follow Responsible AI best practices throughout AI lifecycle.
- Agentic Design
  - Fairness · Explainability · Privacy and security · Safety · Controllability · Veracity and robustness · Governance · Transparency
- Agentic Implementation
  - Filtering mechanisms such as guardrails, principle of least privilege, monitoring for behavior drift, tool scanning
- Fully autonomous agents pose increased risks, particularly to safety [paper]

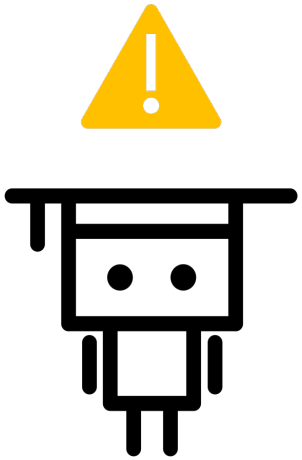
Amazon Bedrock Guardrails



Human in the loop (HITL)

Exceeding failure thresholds

Agent retries exceeded at tool or task level



High-risk or final decisions

Sensitive, irreversible, consequential, or critical actions

Key Security Principles (General Best Practices)

- Authenticate every client
  - Use robust authentication mechanisms. Never trust unauthenticated tool calls
- Sandbox tool execution
  - Run tools in isolated containers, validate and sanitize all inputs/outputs
- Least-privilege IAM policies
  - Using AWS accounts, scope roles tightly to what each tool actually needs
- Tool orchestration authorization
  - Enforce deterministic call sequences; don't allow arbitrary tool chaining
- Monitor for definition drift
  - Tools can appear safe in metadata but hide malicious code in implementation

More best practices

- In this and further module's hands-on activities, you will practice implementing Agentic AI solutions using **Amazon Bedrock AgentCore** for code-based labs and **Amazon Quick** for code-free labs
- It is important to understand the safeguards and best practices in place when building Agentic AI systems using these services:
    - Integrate external tools with Amazon Quick Agents using Model Context Protocol (MCP)
    - Amazon Bedrock AgentCore secure memory usage (best practices)
    - Securely connect tools and other resources using AgentCore
    - Security in Amazon Bedrock AgentCore

Responsible AI and Agentic Evaluation

Agentic evaluation

Metrics for agentic evaluation

System-level metrics 🤖	Task-level metrics 📝	Tool-level metrics 🛠️
Measure operational excellence	Measure use-case level performance	Measure API level performance

System-level metrics 🤖

Some examples are:

- Latency per tool call
- Task completion time
- API call frequency
- Cost and token usage per task
- LLM call error rate

Task-level metrics 

Some examples are:

- Agent success rate
- Task completion rate
- Human intervention rate

Tool-level metrics 🛠️

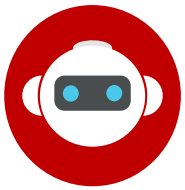
Some examples are:

- ✓ Tool selection accuracy
- Tool parameter call accuracy
- Tool success rate

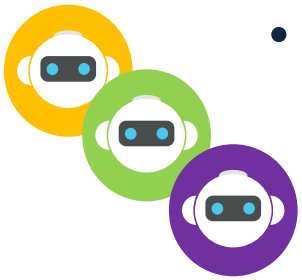
Agentic Orchestration Patterns and Landscape

Agentic orchestration patterns

Orchestration patterns

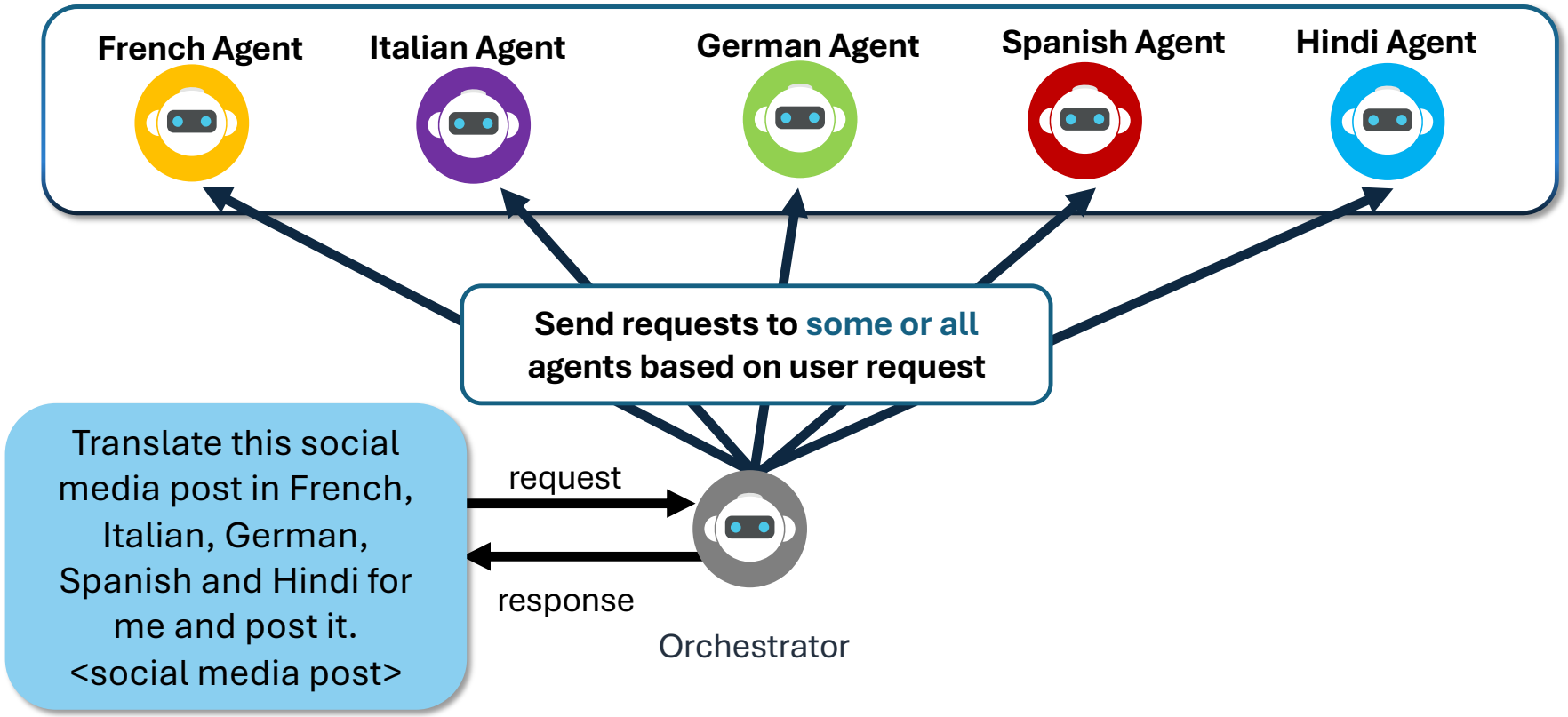


- Single Agent Systems
  - standalone agents operating independently

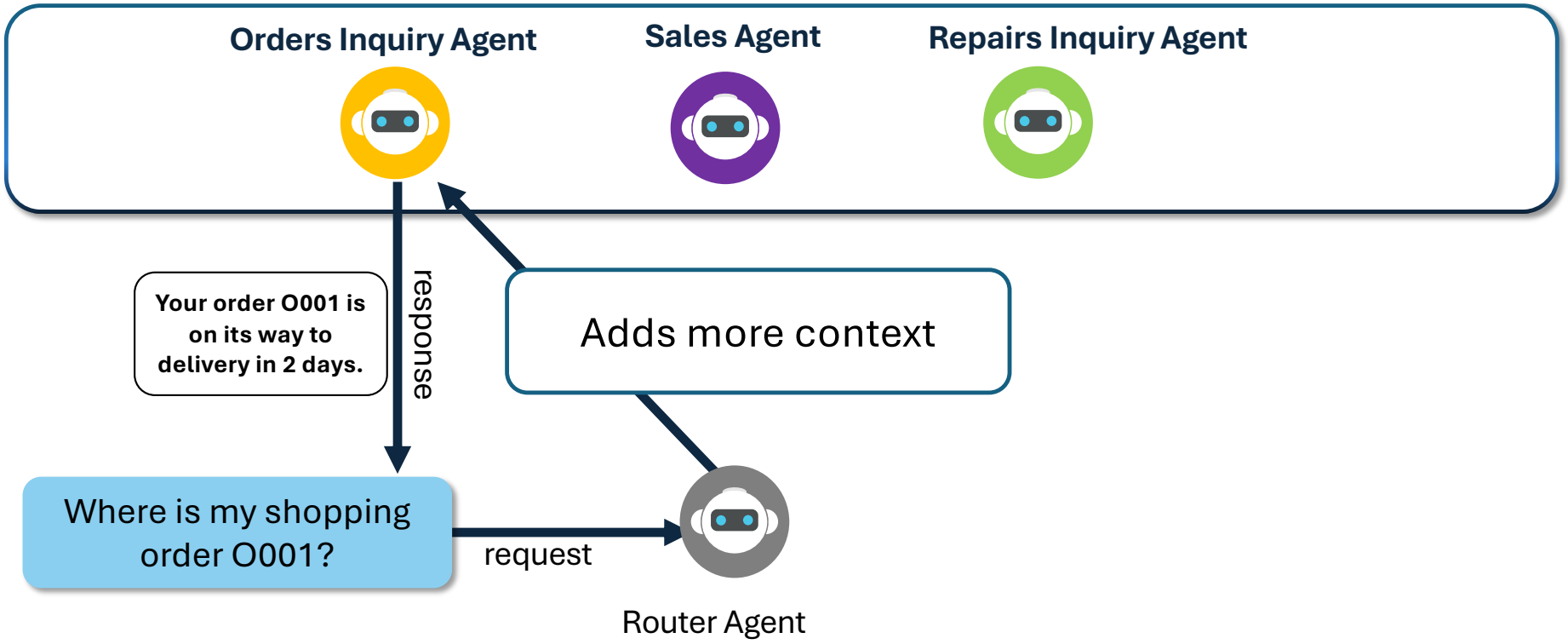


- Multi Agent Systems
  - **Centralized** orchestrator agent
  - **Decentralized** router or peer-based agents

Orchestrator agents



De-centralized agents



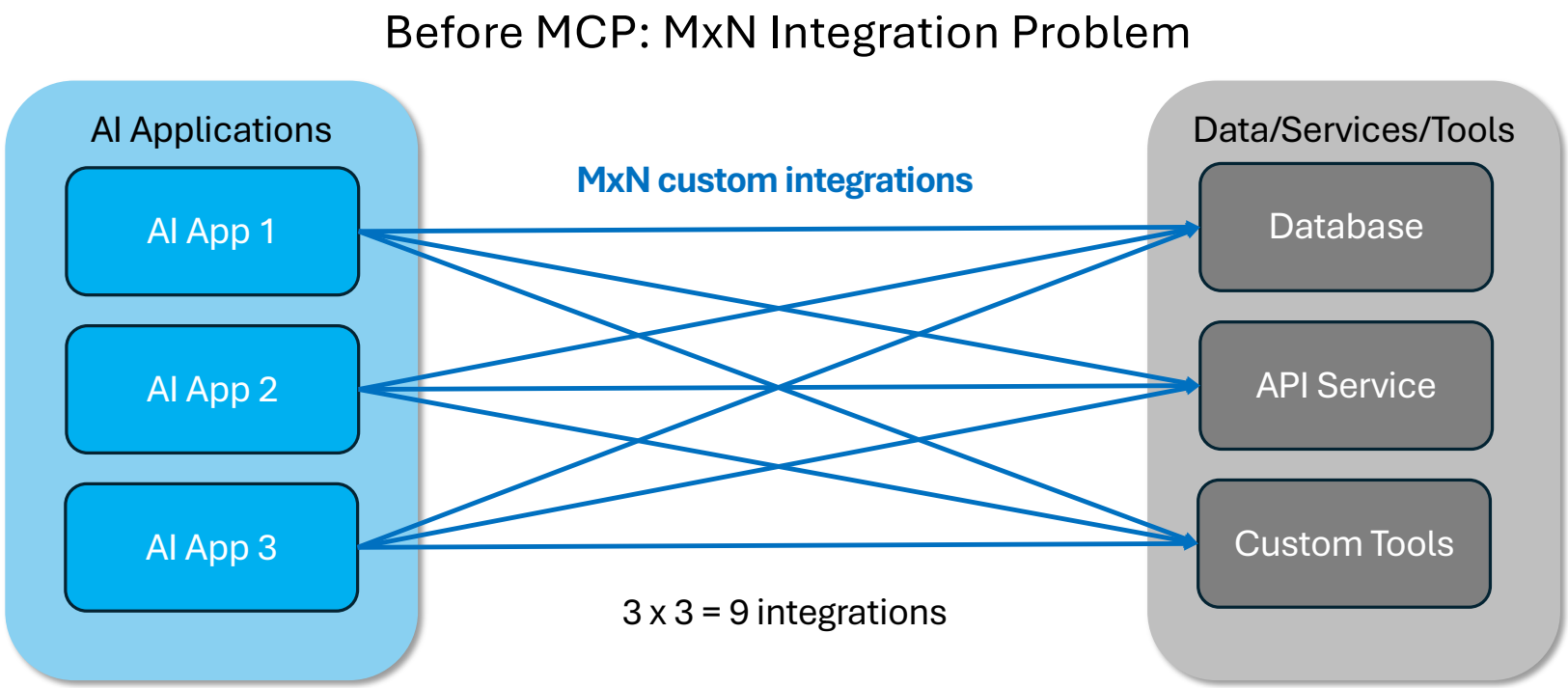
Agentic Orchestration Patterns and Landscape

Introduction to Model Context Protocol (MCP)

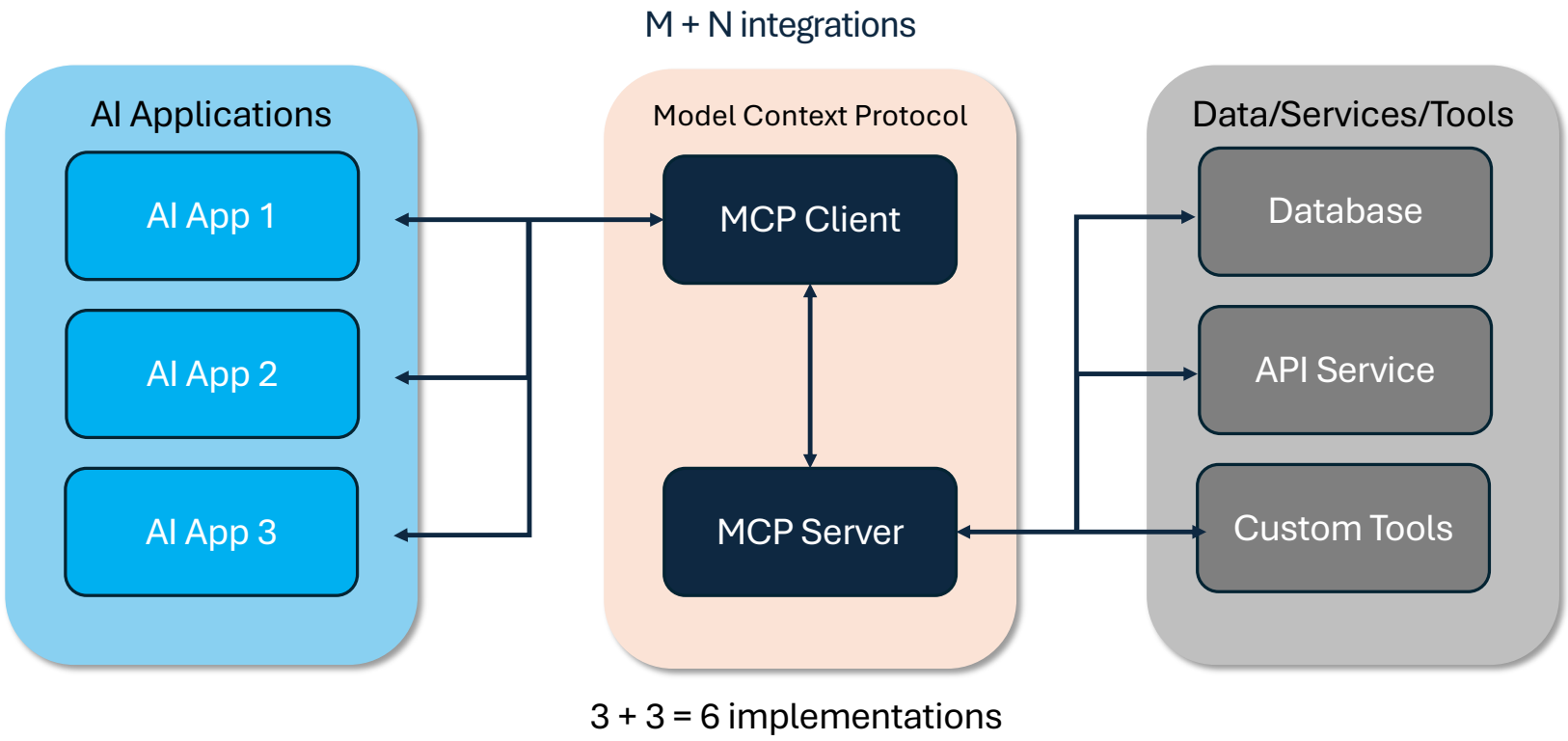
Model Context Protocol (MCP): introduction

- MCP is an open protocol that standardizes how applications provide context to LLMs
- Think of MCP like a USB-C port for AI applications
- It allows builders creating tools independently, free from compatibility issues when integrating with agentic AI systems

Why MCP? N x M problem

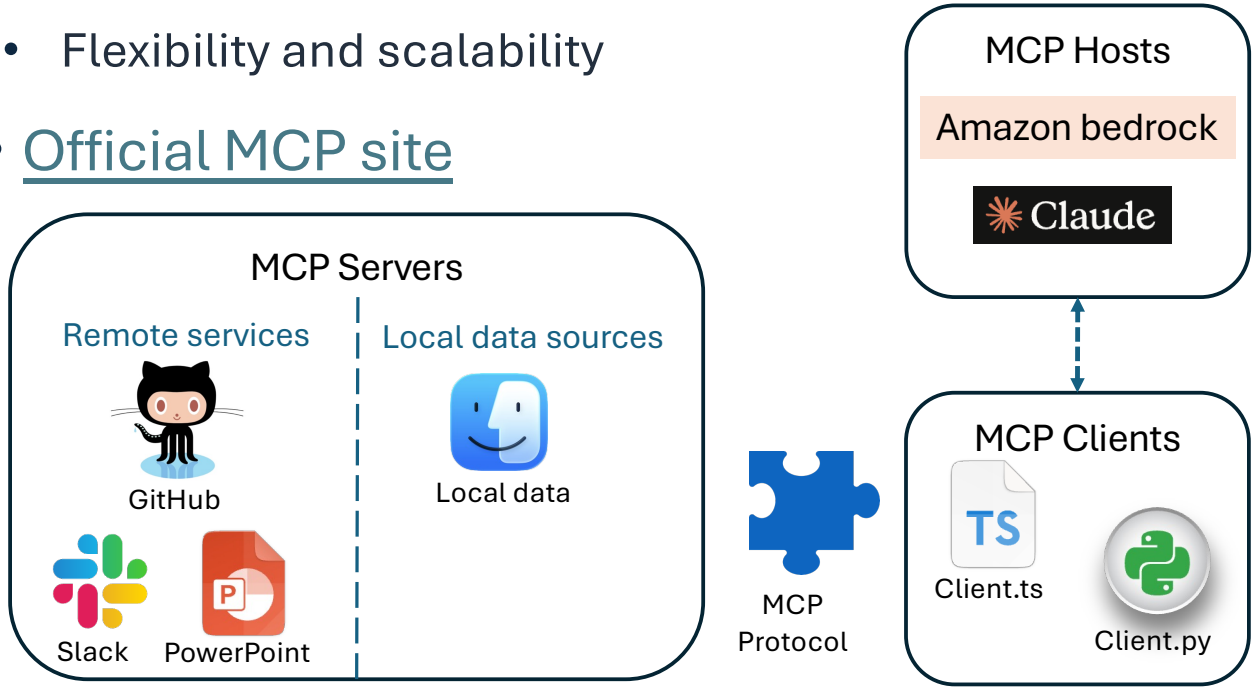


Why MCP? N + M solution



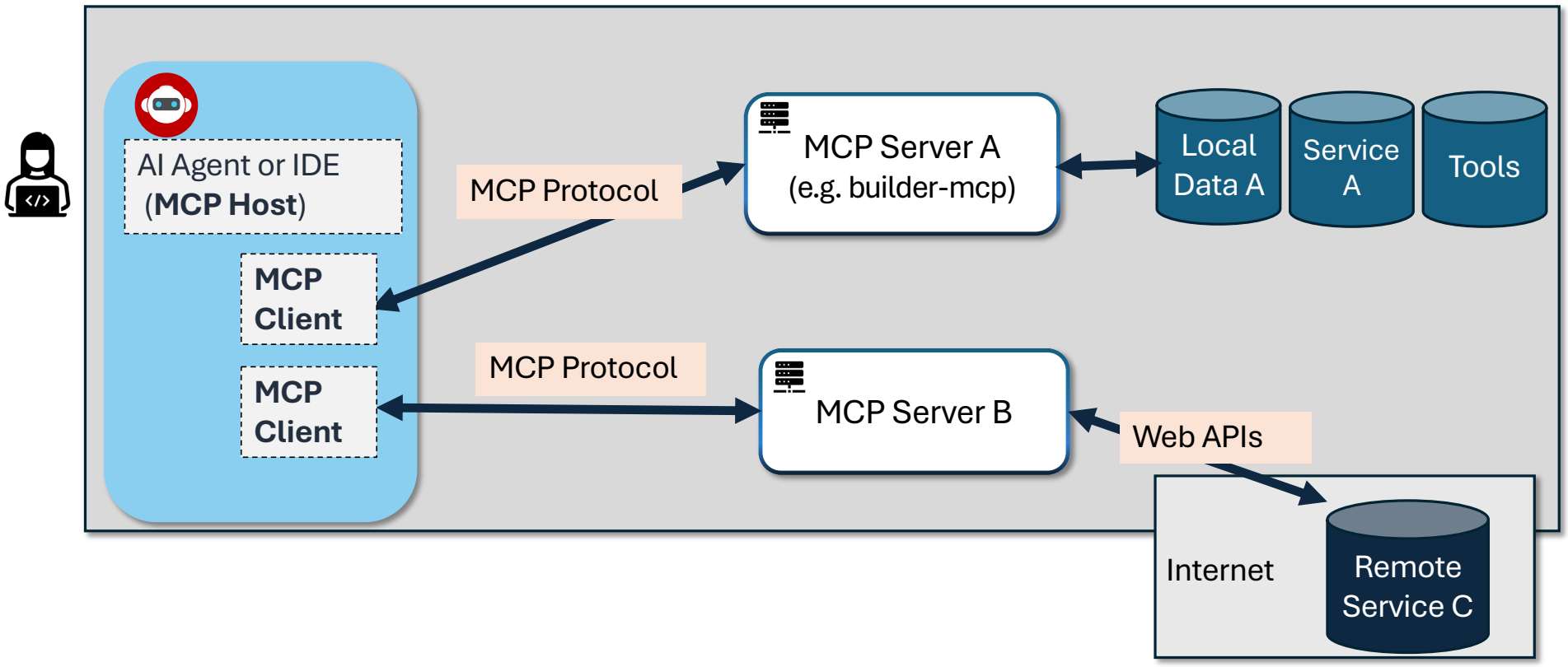
Model Context Protocol (MCP): why use it?

- Problems solved by Model Context Protocol
  - Decouples various tools/data sources from Agentic AI development
  - Standardizes integration with external tools and data sources
  - Flexibility and scalability
- Official MCP site



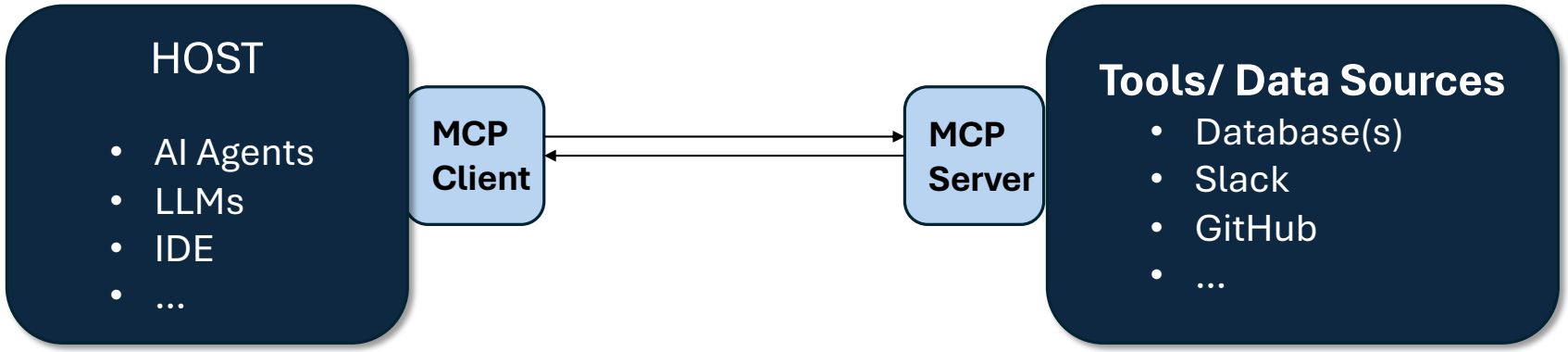
USB port standard also allowed for the independent development of tools guaranteed to connect!

MCP visualized



MCP: components

- Main components of MCP are:
  - Host: AI Agents requiring access to data/tools using MCP
  - MCP client: client software connecting with servers
  - MCP server: Lightweight software exposing capabilities
  - Data sources: Local (database, files) and remote services (APIs) accessible by MCP servers



Agentic Orchestration Patterns and Landscape

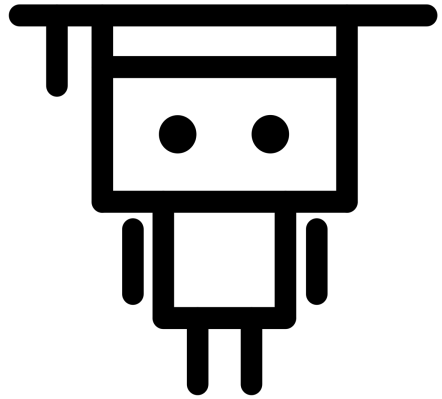
Agentic AI frameworks and landscape

So you want to build an agent?

🧩 No/Low code options

💻 Code-first frameworks

🧬 Specialized frameworks



🌐 Deploy and operate AI agents securely at scale

Agent building – no/low code

- No/low code
  - **Examples:**
    - **Amazon:** PartyRock, Quick ChatAgents
    - N8n, Langflow, Flowwise
  - **When to use:**
    - Rapid prototyping
    - Existing business process automation
    - AI integration

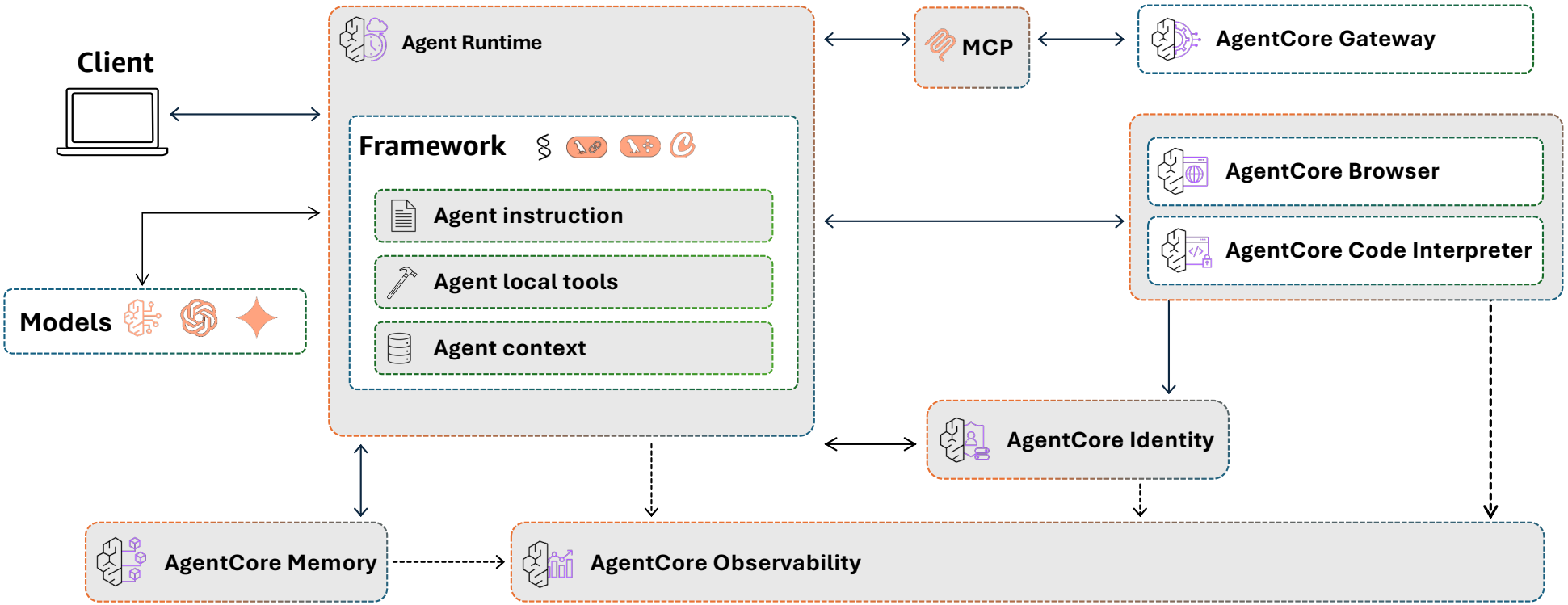
Agent building – code first

- Code-first frameworks
  - **Examples:**
    - Strands Agents, LangGraph, CrewAI, AutoGen
  - **When to use:**
    - Multi-agent orchestration framework
    - Production scale applications
    - Custom agent behavior

Amazon Bedrock AgentCore (agent platform service)

- Scope:
  - Deploy & operate AI agents securely at scale, with any framework & model
- Purpose:
  - Accelerate from prototype to production with fully-managed services that eliminate infrastructure complexity
  - Build agents your way using any framework, model, or tool
  - Deploy agents securely at scale
  - Enhance agents with tools and memory with production grade monitoring
- Impact:
  - See AgentCore home page

Agent building – deploy securely at scale with Amazon Bedrock AgentCore

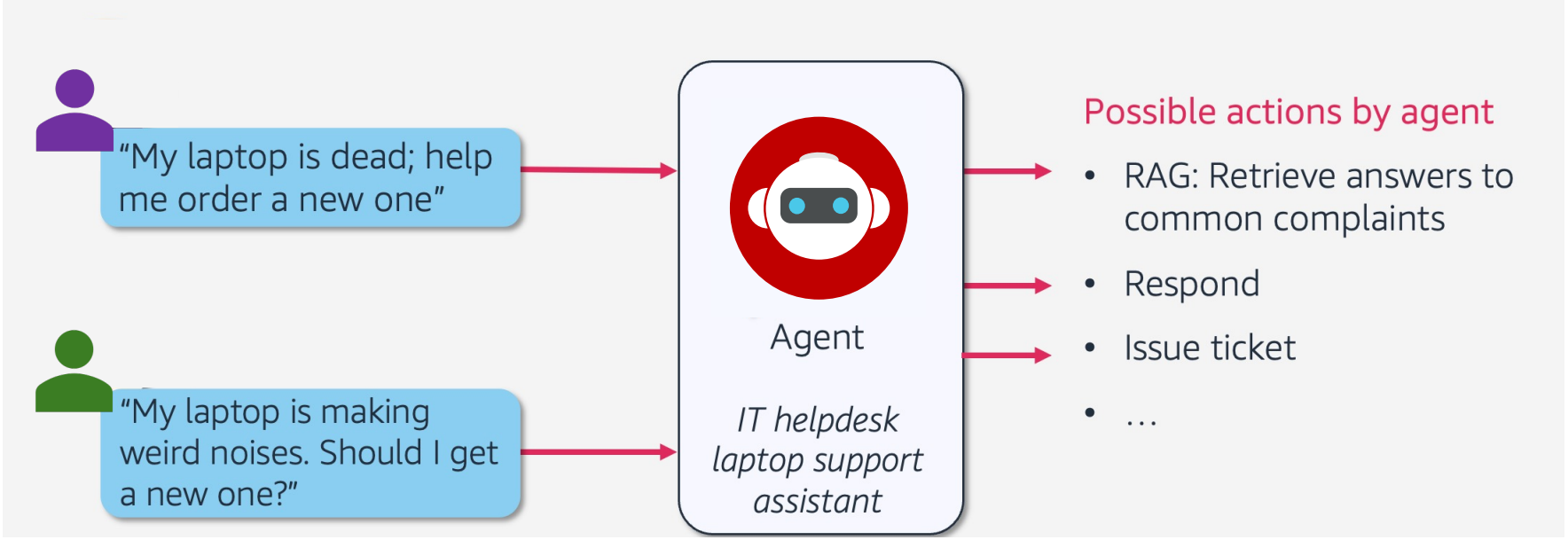


Agentic Orchestration Patterns and Landscape

Agentic AI use cases and application examples

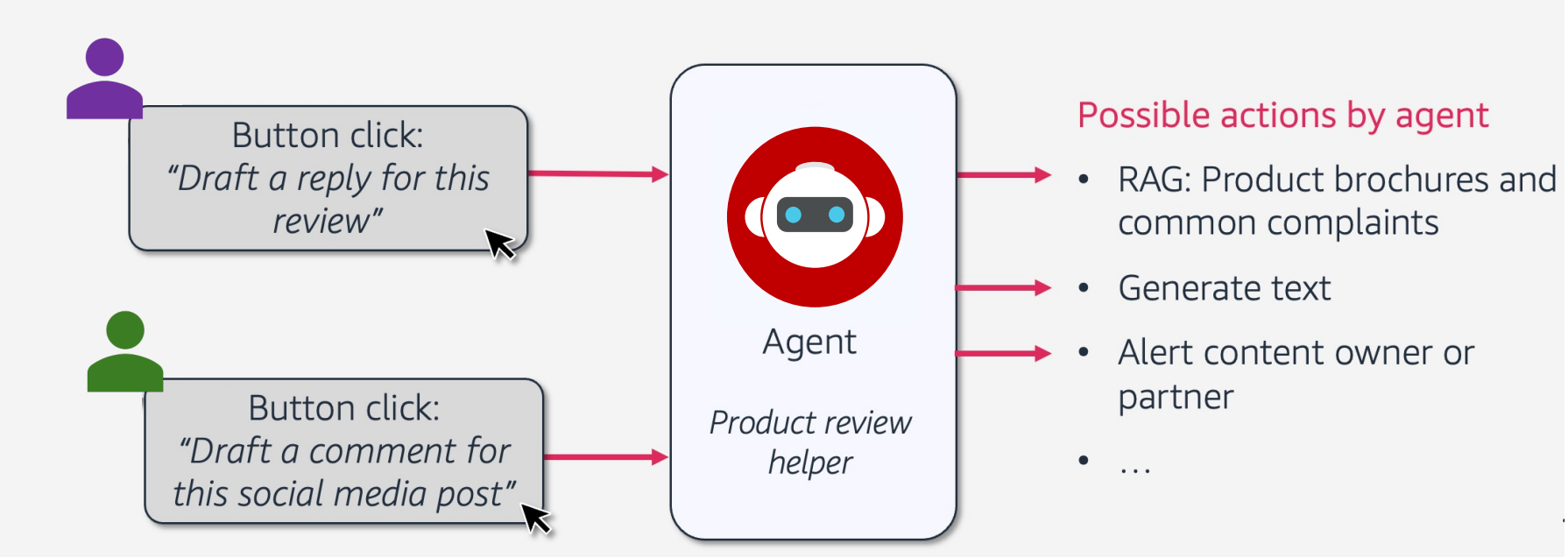
Use case 1: laptop support assistant

- Agents can act as natural language **conversational chatbots** with access to **RAG knowledge** and **tools** that can take some actions



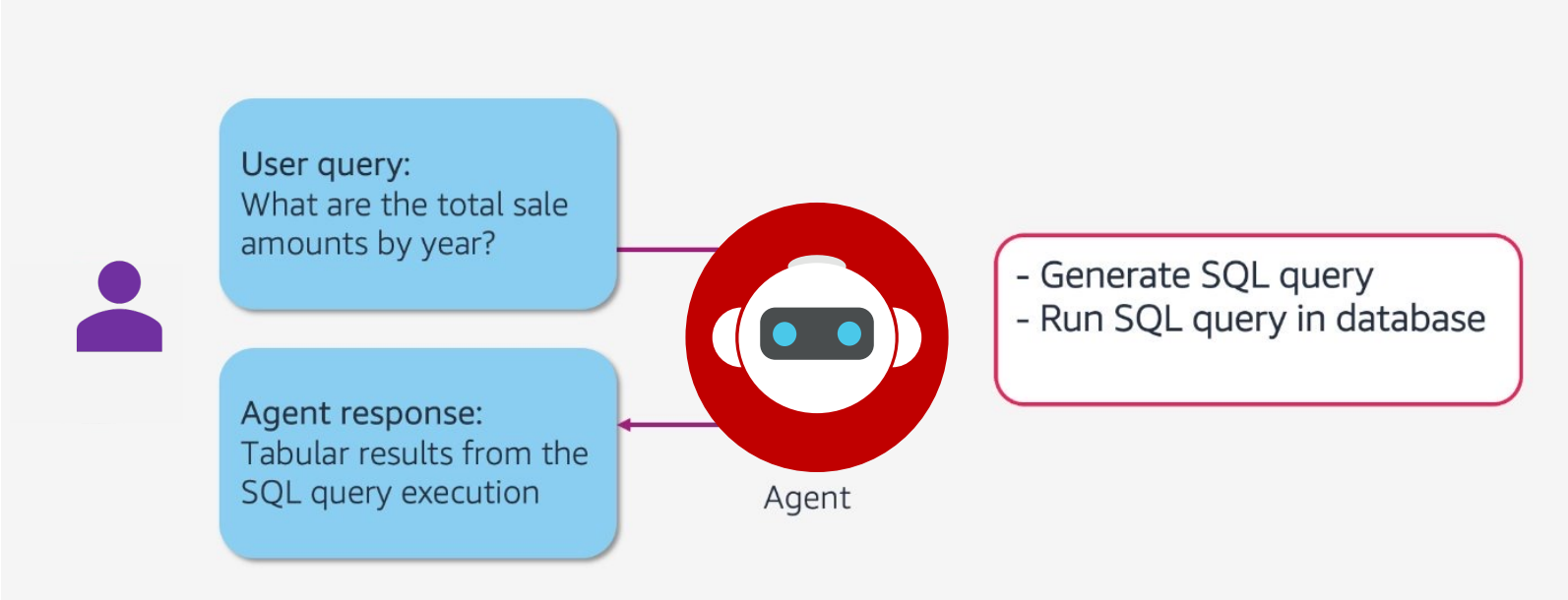
Use case 2: product review helper

- Agents can act as an **event-driven backend** assistant that has access to RAG knowledge and tools, and can take some actions



Use case 3: Text-to-SQL workflow

- LLMs are generally not built to understand and analyze tabular data
- Many use cases involve tables (like finance, accounting, customer analytics)
- Agent can write code and use tools to execute them



Agentic Orchestration Patterns and Landscape

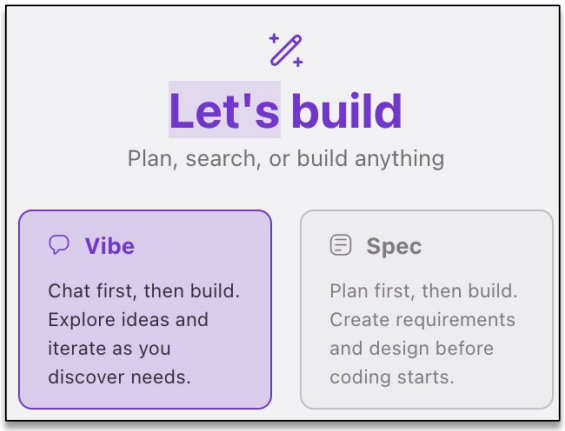
Agents supporting software development

Strands Agents (agent SDK)

- Scope:
  - Strands Agents is a simple-to-use, code-first framework for building agents.
- Purpose:
  - Support flexible configuration options and multiple model providers including Bedrock, Anthropic, Meta, and others.
- Features:
  - Streamlines AI agent development process from local testing to production deployment.
  - Leverages model reasoning to plan, orchestrate tasks, and reflect on goals

Amazon Kiro

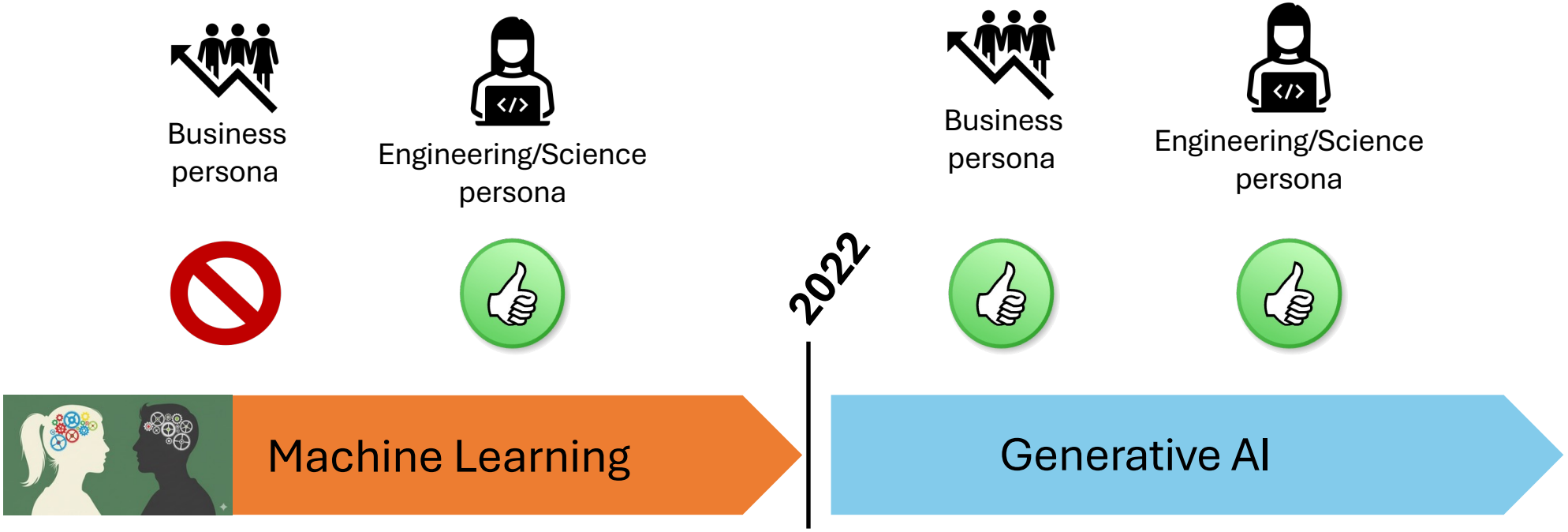
- Scope:
  - Comprehensive development assistance tool
- Purpose:
  - Deliver context-aware assistance directly within integrated development environments
  - Facilitate agentic workflow for code transformation and development
- Key features:
  - Natural prompt to structured requirements
  - Architectural designs backed by best practices
  - Discrete tasks that map to requirements



Agentic Orchestration Patterns and Landscape

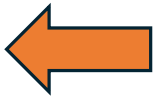
Code-free agentic AI

AI visibility (Users vs Builders)



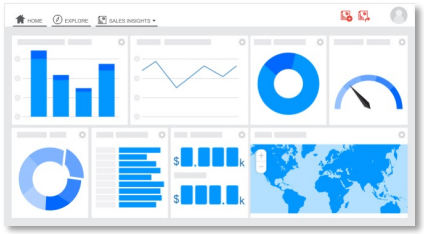
How do you fit as a code-free persona?

business persona

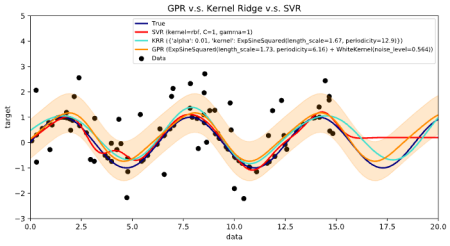


2022

Accessing BI data



Using ML Predictions



Computer assistance needed!

Code-free personas are builders now!

2022

business persona

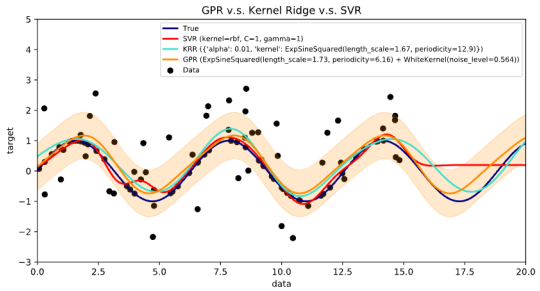
Create a dashboard with sales results for Q1 2025 with the following information...

Create an app to predict customer churn based on historical data from ...

Schedule my project plan meeting next 10am next Wednesday, invite John, Mary,...

Accessing BI data

Using ML Predictions



My personal tasks

Amazon Quick – Building code-free assistants

What is Amazon Quick?

- Amazon Quick is an enterprise AI platform combining generative AI-powered search, deep research over company data, intelligent agents and automations, and business intelligence
- Some example use cases:
  - It can use agents to search global databases, summarize papers, generate citations, and support grant writing, reducing faculty workload and accelerating publication
  - Agents can help generating and update lessons, quizzes, and multimedia aligned to curriculum standards, personalizing learning experiences

Amazon Quick – Chat Agents

- For this hands-on lab, students will begin building chat agents
- Chat agents help students explore data, analyze information, and take actions
- Students can interact with chat agents using the Quick chat interface:
    - to provide assistance through open-ended conversations supported by specific goals, sources of knowledge, and any connected tools
    - to evolve from simple question-answering interfaces to more advanced functions that orchestrate complex workflows

Amazon Quick – Spaces

- A space in Amazon Quick is a collection of data and Quick resources scoped for a particular team or domain
- Spaces allow you to get the most relevant results from conversational agents and other AI tools by grounding the results with only data relevant for your task or domain
- You peers can contribute to the knowledge inside a space so everyone can share the same data

Lab 1.1: Creating a custom agent chat

Practice creating custom a chat agent in Amazon Quick, to build an AI-powered tool that supports academic study activities

Hands-on lab: Working with your study plan

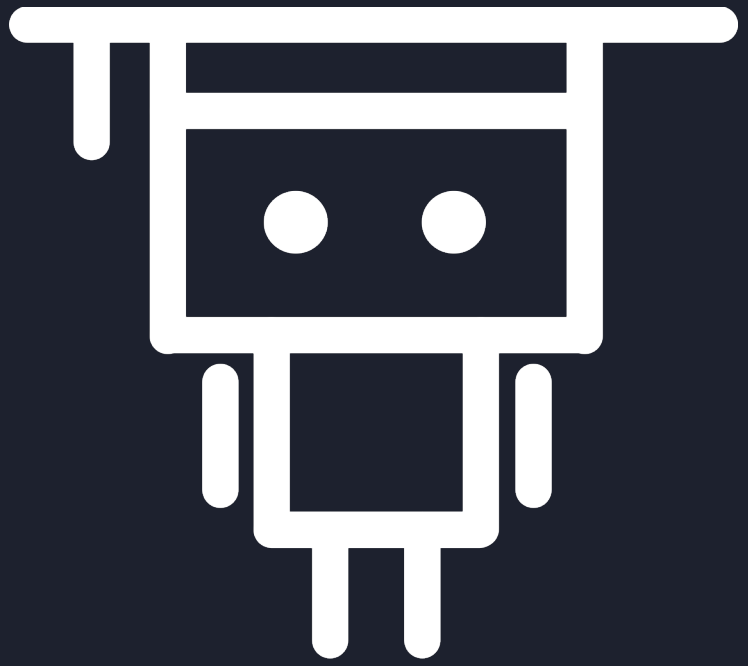
- What do you need for the lab?
  - Access to Amazon Quick
  - Follow the instructions in MLU-EEP-M1-Lab1.1-Quick hands-on guide.pdf
- In this first hands-on lab, you will be using Amazon Quick to build an AI-powered study companion to search notes, research topics, automate tasks, and unlock insights to help you plan your studies.
- You will explore how to customize a Quick chat agent for your studies and will start building your own Quick Space with the data relevant to your academic study domain

Conclusion

Key takeaways

Key takeaways

- AI agents can do more than produce language, by leveraging tools and APIs
- They can understand natural language, generate a plan, identify necessary tools, execute the tools, and overcome obstacles by retrying
- Consider evaluating the pros, cons and the need to build or migrate to Agentic AI workflows for your business use-cases.
  - Consider the use cases, model choice, agentic AI framework, security and compliance considerations, responsible AI considerations, and proof of value before development and deployment
- Agentic evaluation requires careful monitoring and summarization of system, task and tool level metrics.
- You can leverage internal AI agents today!



Thank you!